OpenTelemetry Arrow

Microsoft Observability

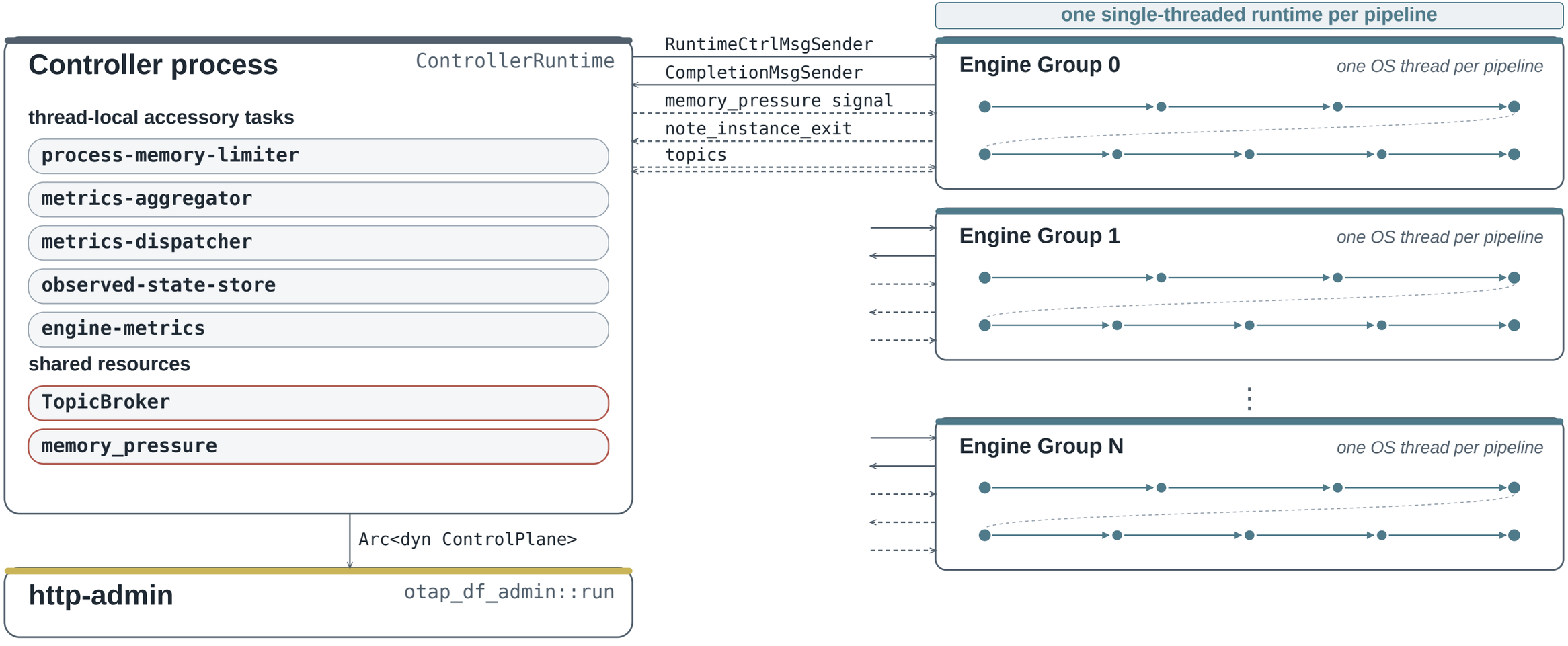
Joshua MacDonald <jmacdonald@microsoft.com>

May 11, 2026

OpenTelemetry Arrow Dataflow Engine

`ControllerRuntime + N × g × RuntimePipeline`

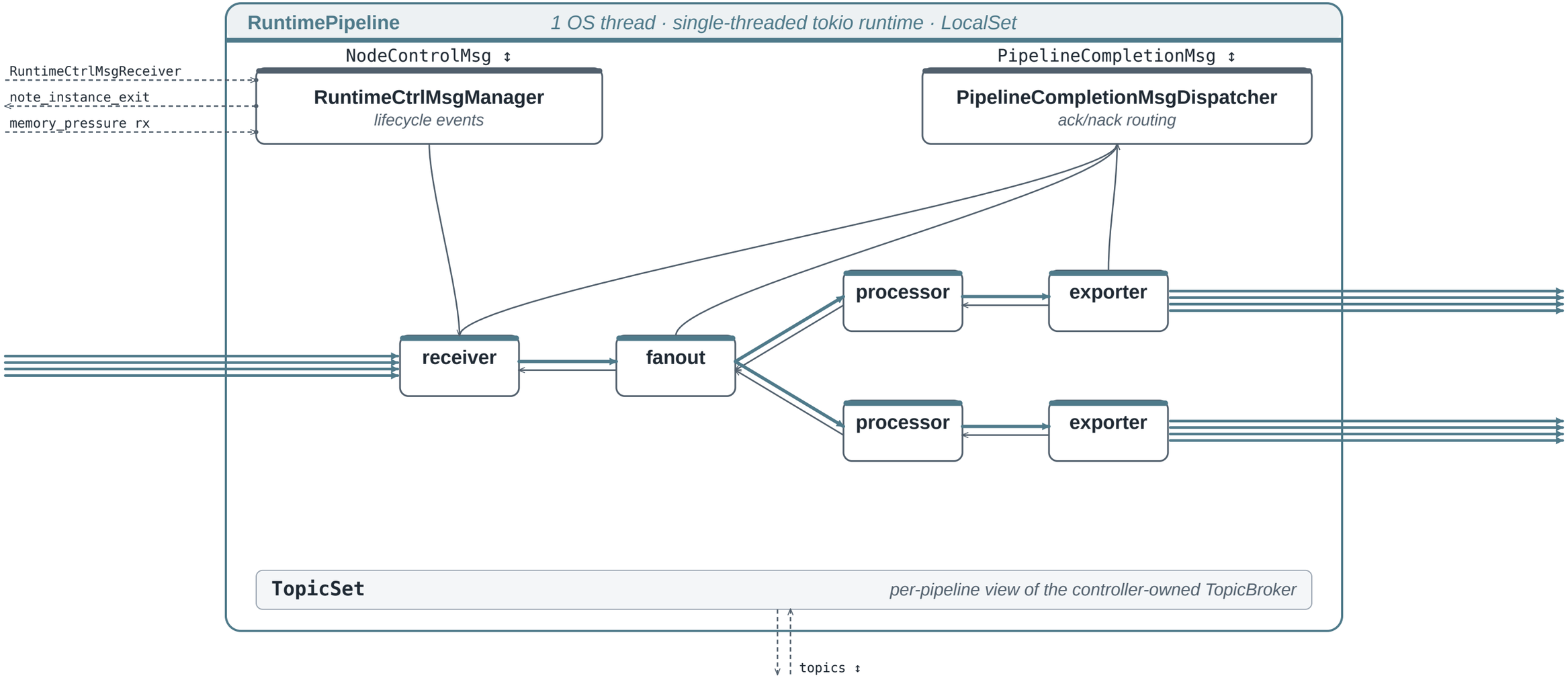
Single controller, one engine per CPU, one thread per pipeline.



Pipeline Engine

RuntimePipeline

Thread-per-core. Single-threaded async runtime.



Pipeline Data

Context + payload

Pipeline data is payload and context, each carries its own callstack

OtapPdata

Frame *top*
node_id = 7
interests Ack | Nack | Producer

⋮

Frame *bottom*
node_id = 1
interests Ack | Nack | Return

transport_headers	Option<TransportHeaders>
flow_compute_ns	Option<NonZeroU64>

OtapArrowRecords OTAP

OTAP records

reference-counted · immutable

one of

OtlpBytes OTLP

OTLP bytes

reference-counted · immutable

Frame

node_id	usize
interests	Interests
route.calldata	CallData = SmallVec<[Context8u8; 3]>
route.entry_time_ns	u64
route.output_port_index	u16

Interests

ACKS	NACKS	RETURN_DATA	ENTRY_TIMESTAMP
1<<0	1<<1	1<<2	1<<3
CONSUMER_METRICS	PRODUCER_METRICS	SOURCE_TAGGING	PROCESS_DURATION
1<<4	1<<5	1<<6	1<<7

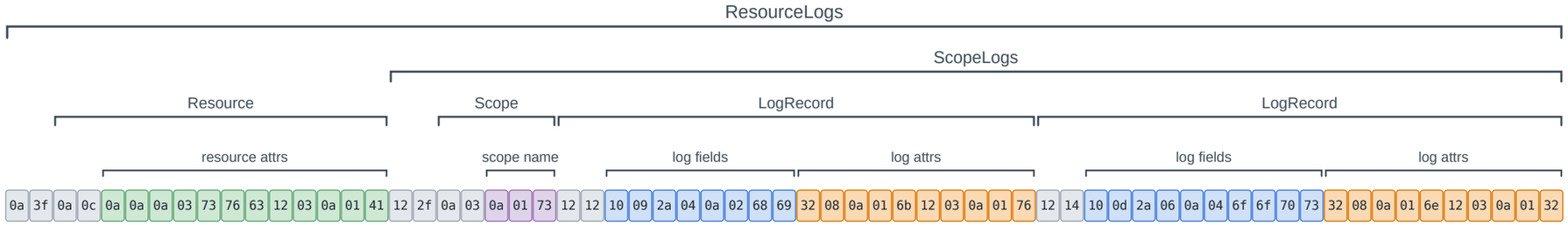
OTLP Logs

protocol buffer bytes

OTLP structure LogsData → ResourceLogs → ScopeLogs → LogRecord

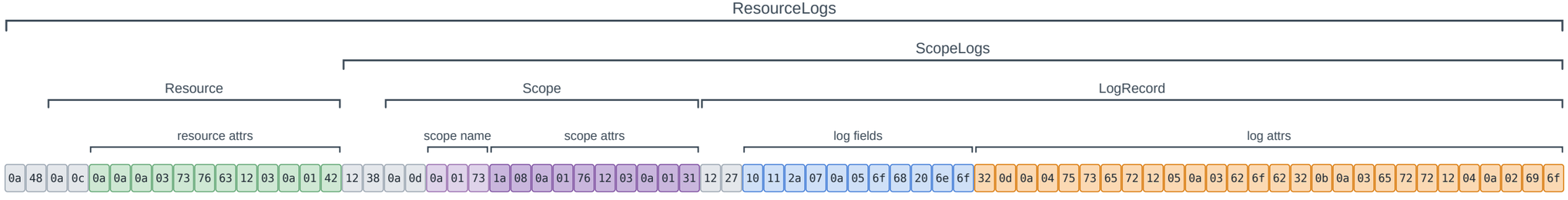
Batch A

65 bytes on the wire



Batch B

74 bytes on the wire



Message frame · Resource attrs · Scope name · Scope attrs · LogRecord fields · Log attrs

OTAP Logs

4 x RecordBatch

Row-oriented OTLP becomes columnar Arrow tables joined by id.

resource_attrs

parent_id	key	str
0	svc	A
1	svc	B

scope_attrs

parent_id	key	str
1	v	1

log_attrs

parent_id	key	str
0	k	v
1	n	2
2	user	bob
2	err	io

logs

id	resource	scope	scope.name	severity	body
0	0	0	s	9	hi
1	0	0	s	13	oops
2	1	1	s	17	oh no

Source OTLP records

Batch A

```
0a 3f 0a 0c 0a 0a 0a 03 73 76 63 12 03 0a 01 41 12 2f 0a 03 0a 01 73 12 12 10 09 2a 04 0a 02 68 69 32 08 0a 01 6b 12 03 0a 01 76 12 14 10 0d 2a 06 0a 04 6f 6f 70 73 32 08 0a 01 6e 12 03 0a 01 32
```

Batch B

```
0a 48 0a 0c 0a 0a 0a 03 73 76 63 12 03 0a 01 42 12 38 0a 0d 0a 01 73 1a 08 0a 01 76 12 03 0a 01 31 12 27 10 11 2a 07 0a 05 6f 68 20 6e 6f 32 0d 0a 04 75 73 65 72 12 05 0a 03 62 6f 62 32 0b 0a 03 65 72 72 12 04 0a 02 69 6f
```

Identifier · Resource attrs · Scope name · Scope attrs · LogRecord fields · Log attrs

Pipeline Data

Context + payload

Pipeline data is payload and context, each carries its own callstack

OtapPdata

Frame *top*
node_id = 7
interests Ack | Nack | Producer

⋮

Frame *bottom*
node_id = 1
interests Ack | Nack | Return

transport_headers	Option<TransportHeaders>
flow_compute_ns	Option<NonZeroU64>

OtapArrowRecords OTAP

OTAP records

reference-counted · immutable

one of

OtlpBytes OTLP

OTLP bytes

reference-counted · immutable

Frame

node_id	usize
interests	Interests
route.calldata	CallData = SmallVec<[Context8u8; 3]>
route.entry_time_ns	u64
route.output_port_index	u16

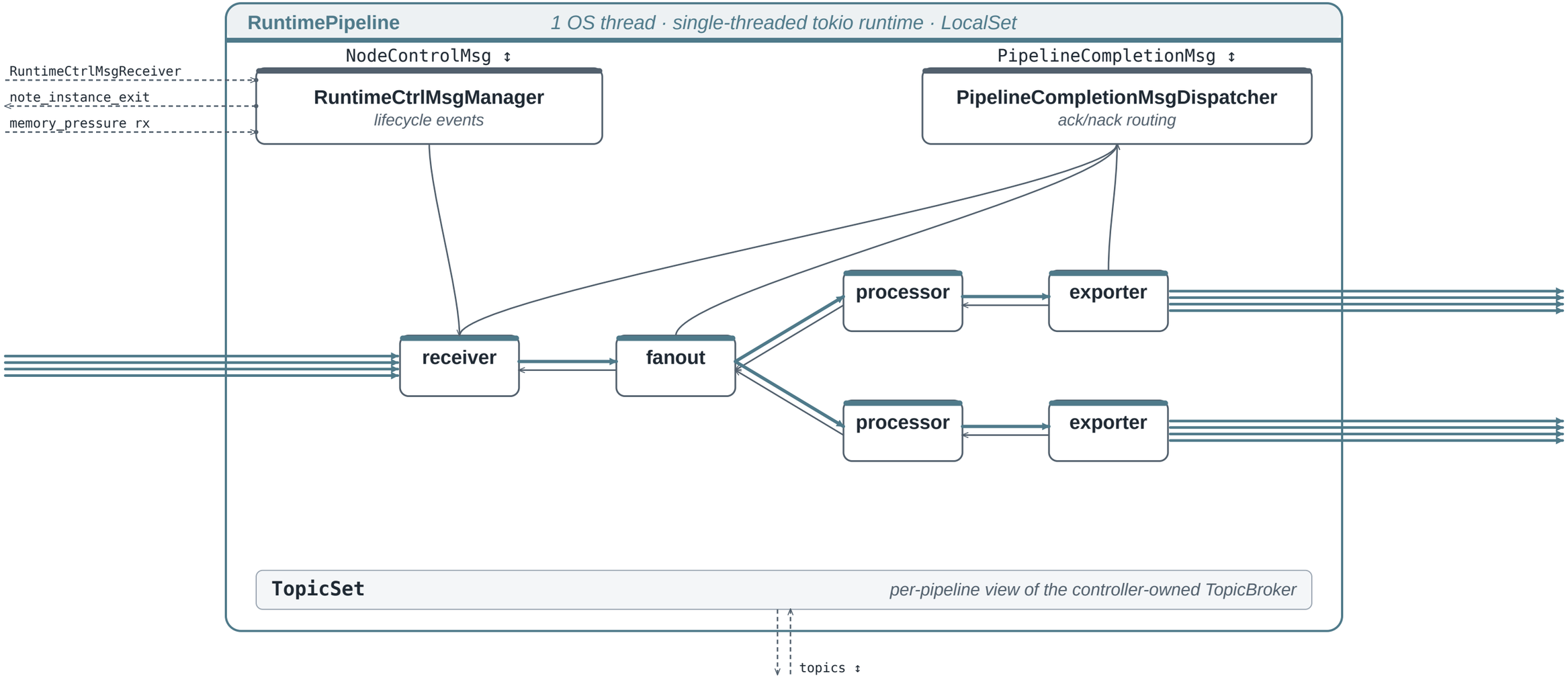
Interests

ACKS	NACKS	RETURN_DATA	ENTRY_TIMESTAMP
1<<0	1<<1	1<<2	1<<3
CONSUMER_METRICS	PRODUCER_METRICS	SOURCE_TAGGING	PROCESS_DURATION
1<<4	1<<5	1<<6	1<<7

Pipeline Engine

RuntimePipeline

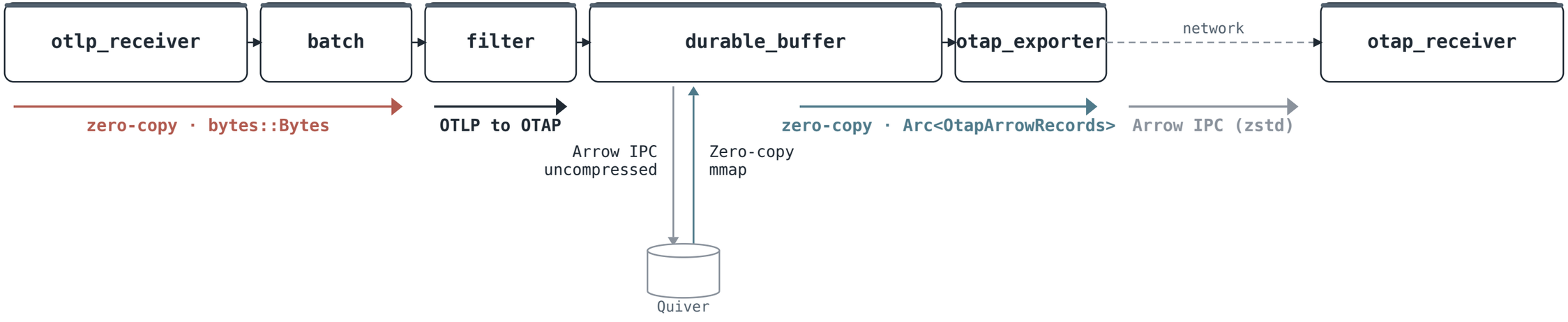
Thread-per-core. Single-threaded async runtime.

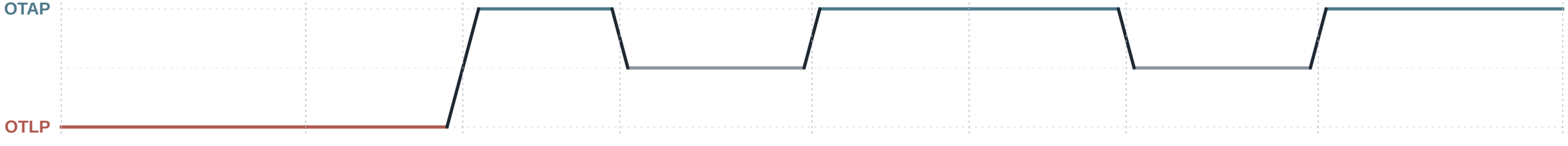


Zero-copy & Arrow IPC

signal level view

OTLP to OTAP on demand, Arrow IPC serialization





Memory and backpressure

`MemoryPressureState + bounded per-node state`

Queues and maps are fixed size, governed by process memory-limiter.

Controller process

`ControllerRuntime`

process-memory-limiter

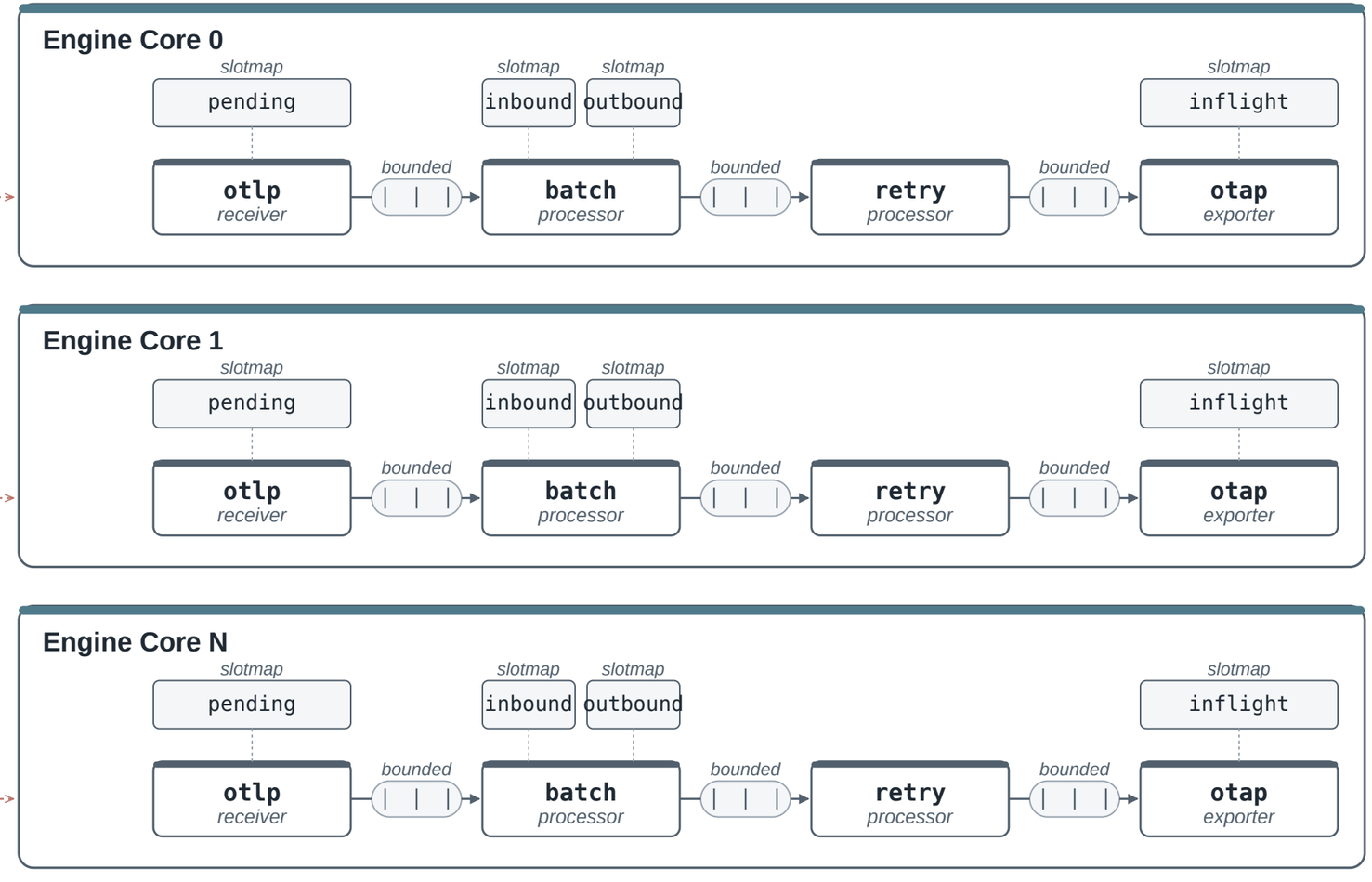
MemoryPressureState	*Arc<…Inner>*
level	AtomicU8
usage_bytes	AtomicU64
soft_limit_bytes	AtomicU64
hard_limit_bytes	AtomicU64
retry_after_secs	AtomicU32
mode	AtomicU8
fail_readiness_on_hard	AtomicBool

Normal | Soft | Hard

MemoryLimiterPolicy

mode	MemoryLimiterMode
source	MemoryLimiterSource
check_interval	Duration
soft_limit / hard_limit	Option<u64>
hysteresis	Option<u64>
retry_after_secs	u32
fail_readiness_on_hard	bool
purge_on_hard	bool

memory_pressure *tokio::sync::watch*



1. **Queues and maps are sized at construction**
   avoid unbounded growth
2. **Backpressure via Acks/Nacks and calldata**
   unwind call stack using subscription state
3. **Hard memory pressure signals ingress to stop**
   receivers reject with Retry-After

Roadmap

Where OpenTelemetry Arrow is going.

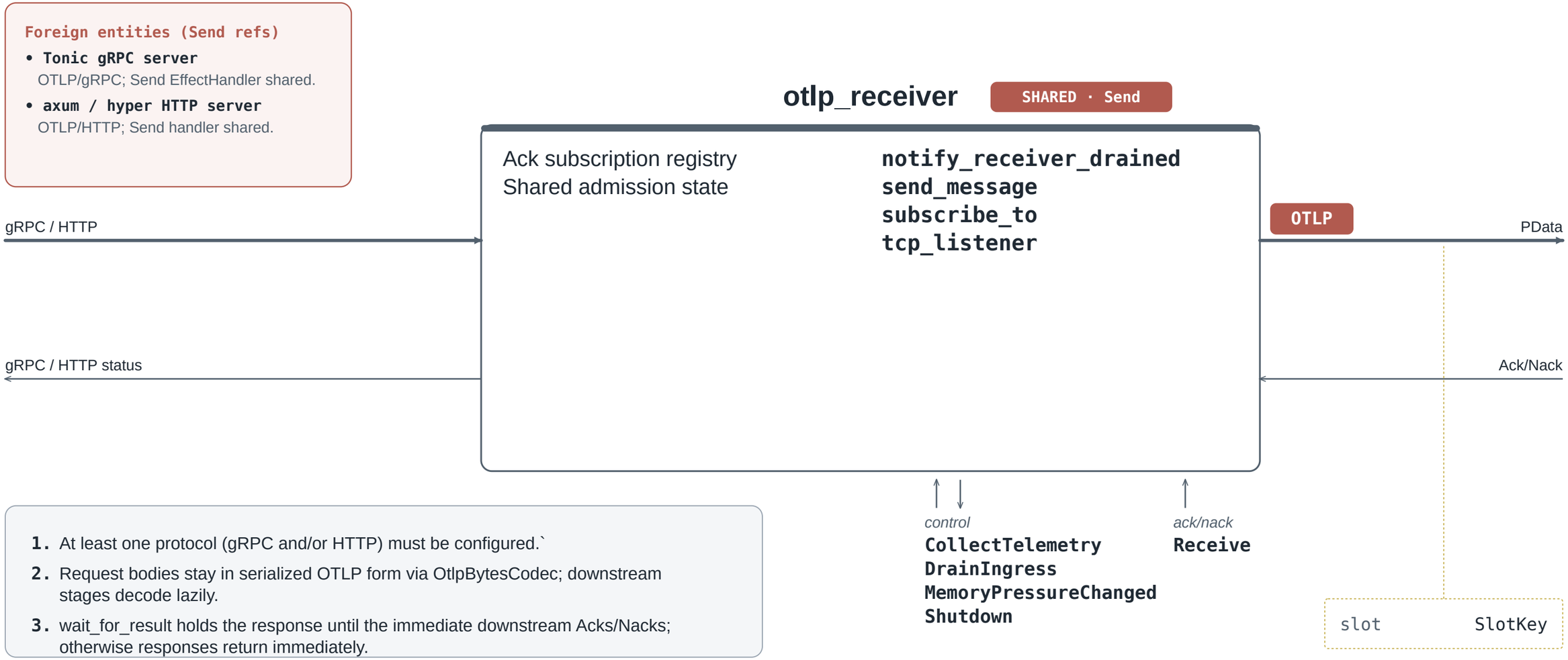
- Naming
- Extensions
- Binary Distro
- Multi-tenant
- Plugins (Web Assembly)
- NUMA configuration
- OTel Profiles
- More query languages

otlp_receiver

urn:otel:receiver:otlp

Ingests OTLP/gRPC and OTLP/HTTP exports as OTLP bytes payloads.

protocols.grpc	Option<GrpcServerSettings>
protocols.http	Option<HttpServerSettings>



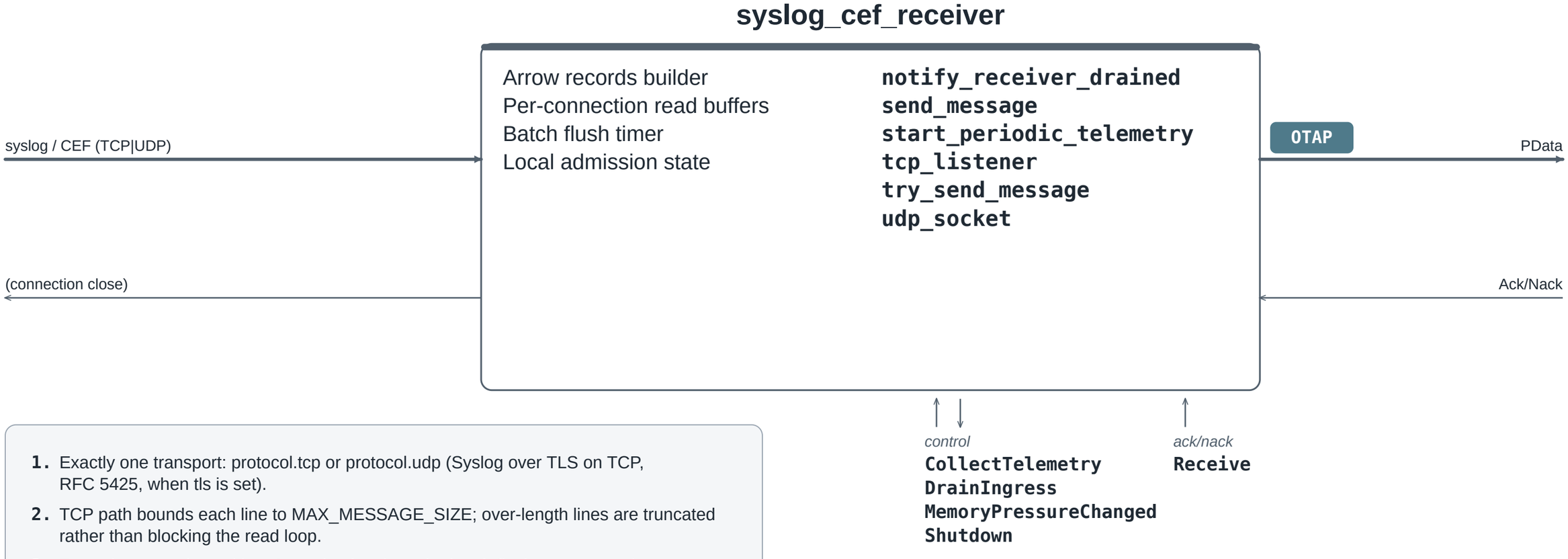
1. At least one protocol (gRPC and/or HTTP) must be configured.`
2. Request bodies stay in serialized OTLP form via OtlpBytesCodec; downstream stages decode lazily.
3. wait_for_result holds the response until the immediate downstream Acks/Nacks; otherwise responses return immediately.
4. max_concurrent_requests defaults to 0 = adopt the downstream pdata channel

syslog_cef_receiver

`urn:otel:receiver:syslog_cef`

Listens on TCP or UDP for RFC 5424 syslog and ArcSight CEF messages; emits accumulated OTAP log batches downstream.

protocol	Protocol (Tcp|Udp)
protocol.tcp.listening_addr	SocketAddr
protocol.tcp.tls	Option<TlsServerConfig>
protocol.udp.listening_addr	SocketAddr
batch.max_batch_duration_ms	Option<NonZeroU64>
batch.max_size	Option<NonZeroU16>



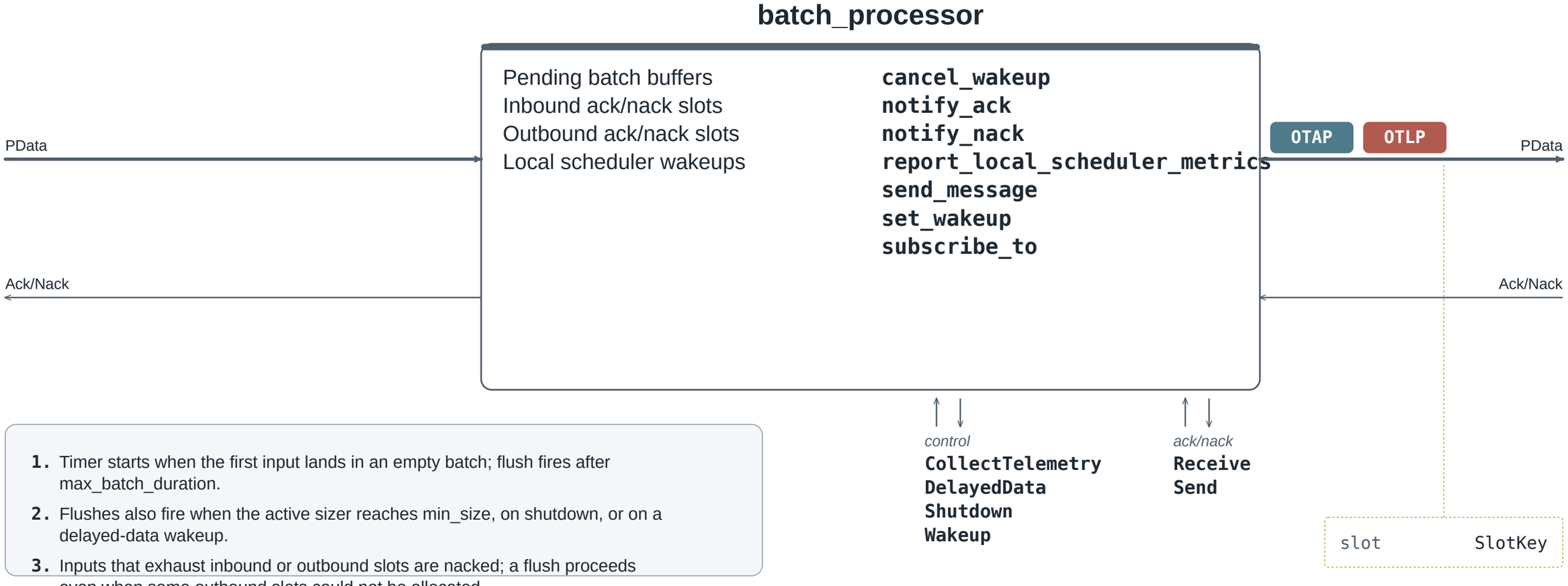
1. Exactly one transport: protocol.tcp or protocol.udp (Syslog over TLS on TCP, RFC 5425, when tls is set).
2. TCP path bounds each line to MAX_MESSAGE_SIZE; over-length lines are truncated rather than blocking the read loop.
3. Records are coalesced into Arrow log batches; the batch flushes on max_size or max_batch_duration_ms (defaults: 100 records / 100 ms).

batch_processor

urn:otel:processor:batch

Coalesces inputs into size-bound batches with a max-age timer.

otap	FormatConfig
otlp	FormatConfig
max_batch_duration	Duration
inbound_request_limit	NonZeroUsize
outbound_request_limit	NonZeroUsize
format	BatchingFormat



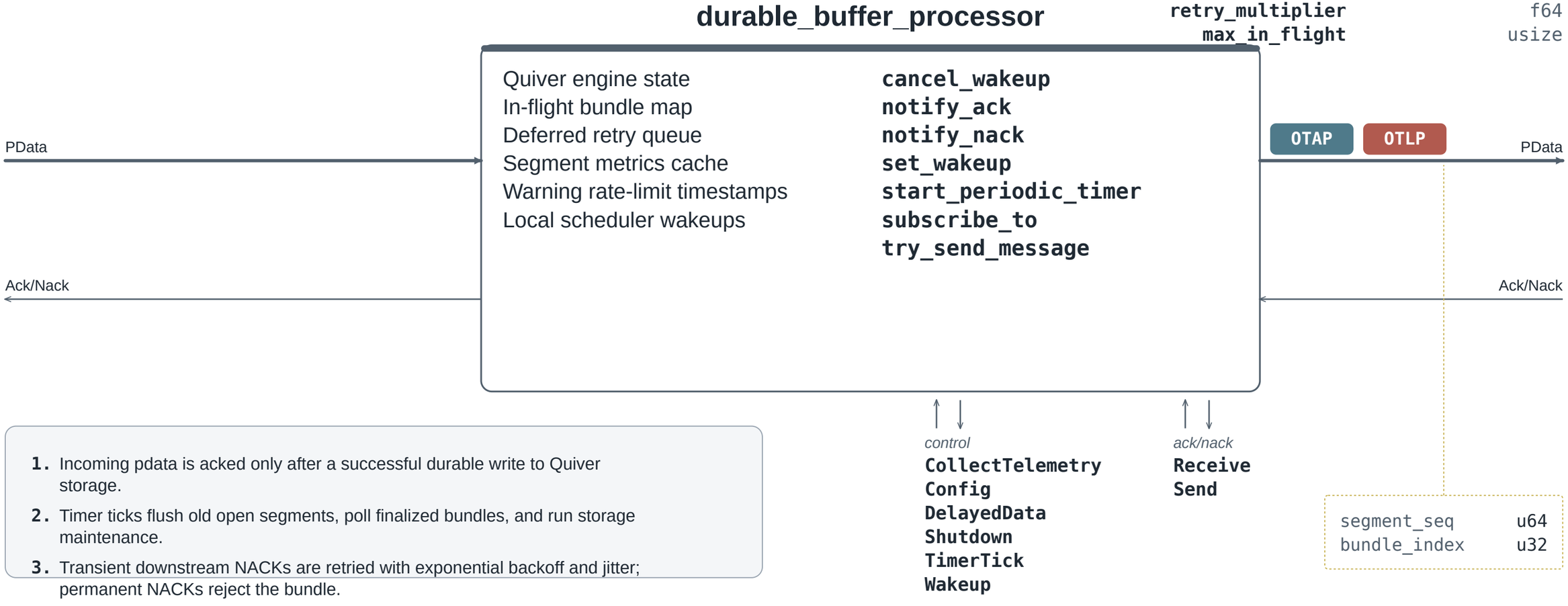
1. Timer starts when the first input lands in an empty batch; flush fires after max_batch_duration.
2. Flushes also fire when the active sizer reaches min_size, on shutdown, or on a delayed-data wakeup.
3. Inputs that exhaust inbound or outbound slots are nacked; a flush proceeds even when some outbound slots could not be allocated.

durable_buffer_processor

`urn:otel:processor:durable_buffer`

Persists pdata to per-core durable storage before forwarding finalized bundles downstream.

path	PathBuf
retention_size_cap	Byte
max_age	Option<Duration>
size_cap_policy	SizeCapPolicy
poll_interval	Duration
otlp_handling	OtlpHandling
max_segment_open_duration	Duration
initial_retry_interval	Duration
max_retry_interval	Duration
retry_multiplier	f64
max_in_flight	usize



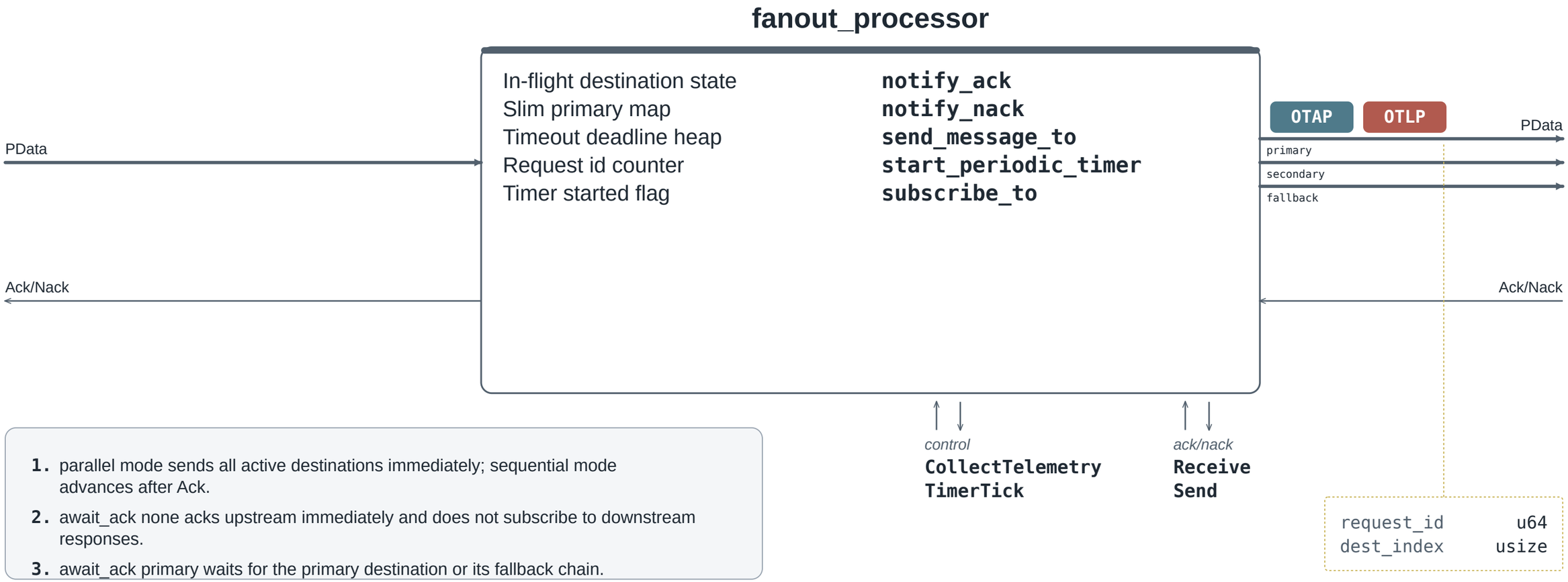
1. Incoming pdata is acked only after a successful durable write to Quiver storage.
2. Timer ticks flush old open segments, poll finalized bundles, and run storage maintenance.
3. Transient downstream NACKs are retried with exponential backoff and jitter; permanent NACKs reject the bundle.

fanout_processor

urn:otel:processor:fanout

Clones each pdata item to configured output ports with optional ack aggregation and fallbacks.

mode	DeliveryMode
await_ack	AwaitAck
destinations	Vec<DestinationConfig>
timeout_check_interval	Duration
max_inflight	usize



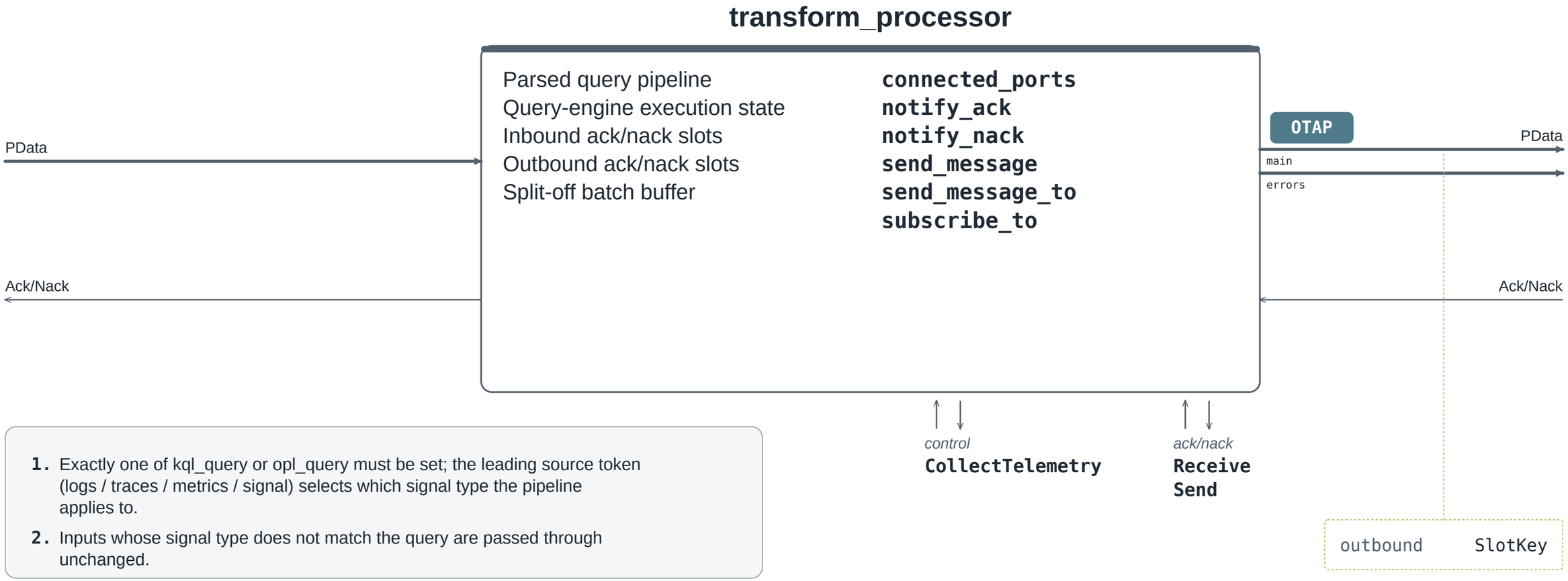
1. parallel mode sends all active destinations immediately; sequential mode advances after Ack.
2. await_ack none acks upstream immediately and does not subscribe to downstream responses.
3. await_ack primary waits for the primary destination or its fallback chain.
4. await_ack all waits for all non-fallback destinations and fail-fast nacks on

transform_processor

urn:otel:processor:transform

Runs a query-engine pipeline over each OTAP batch; the query is written in one of several supported expression languages.

kql_query	String
opl_query	String
inbound_request_limit	NonZeroUsize
outbound_request_limit	NonZeroUsize
skip_sanitize_result	bool
filter_attribute_keys_case_sensitive	bool



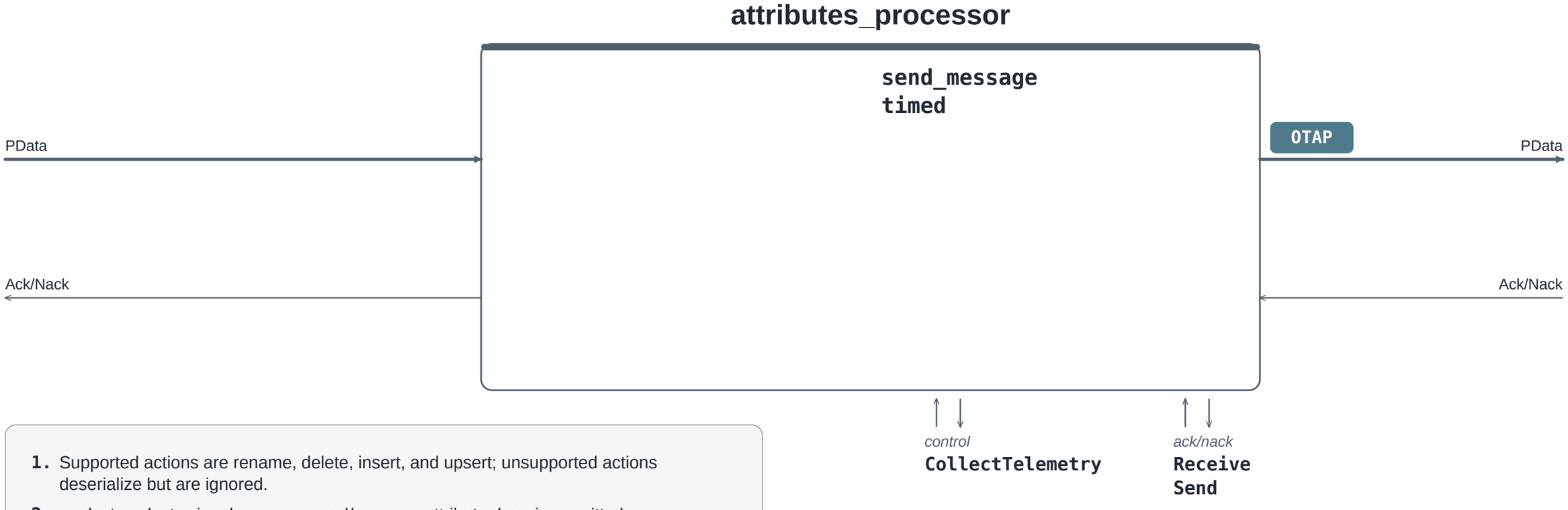
1. Exactly one of kql_query or opl_query must be set; the leading source token (logs / traces / metrics / signal) selects which signal type the pipeline applies to.
2. Inputs whose signal type does not match the query are passed through unchanged.
3. OTLP inputs are converted to OTAP records before the query runs; outputs are emitted as OTAP only.

attributes_processor

urn:otel:processor:attribute

Applies configured attribute edits to selected OTAP attribute domains.

actions	Vec<Action>
apply_to	Option<Vec<String>>



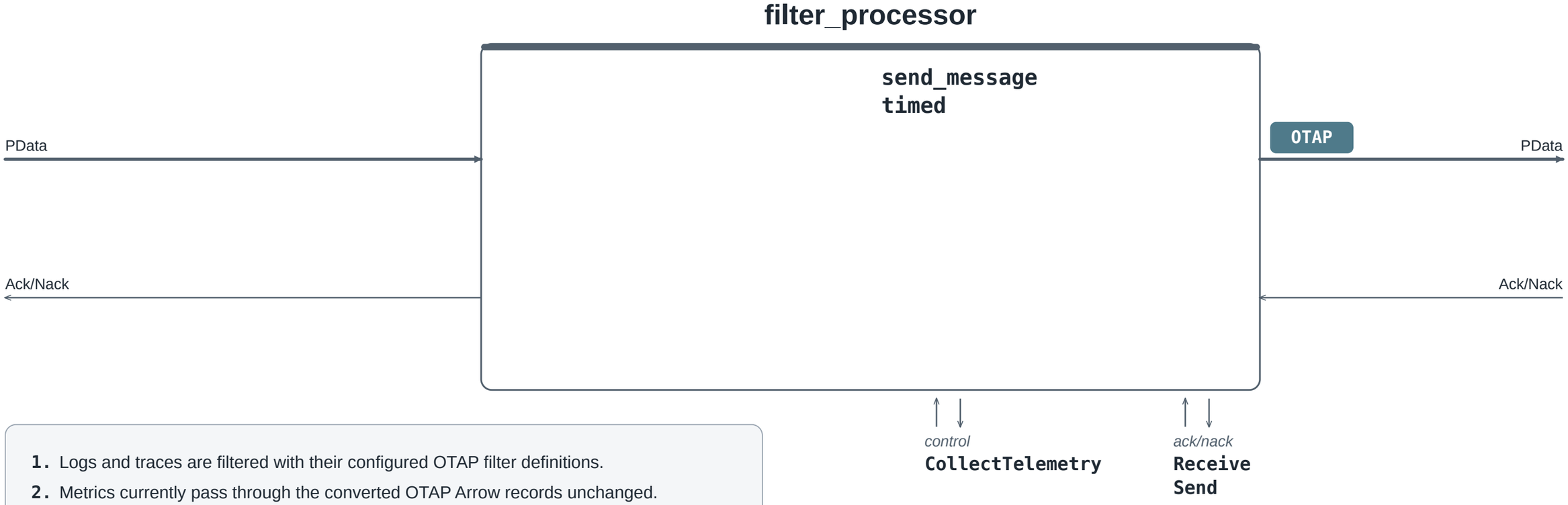
1. Supported actions are rename, delete, insert, and upsert; unsupported actions deserialize but are ignored.
2. apply_to selects signal, resource, and/or scope attribute domains; omitted apply_to defaults to signal attributes.
3. OTLP inputs are converted to OTAP records before transforms run, so non-noop outputs are OTAP.

filter_processor

urn:otel:processor:filter

Applies configured log and trace filters after converting inputs to OTAP Arrow records.

logs	LogFilter
traces	TraceFilter



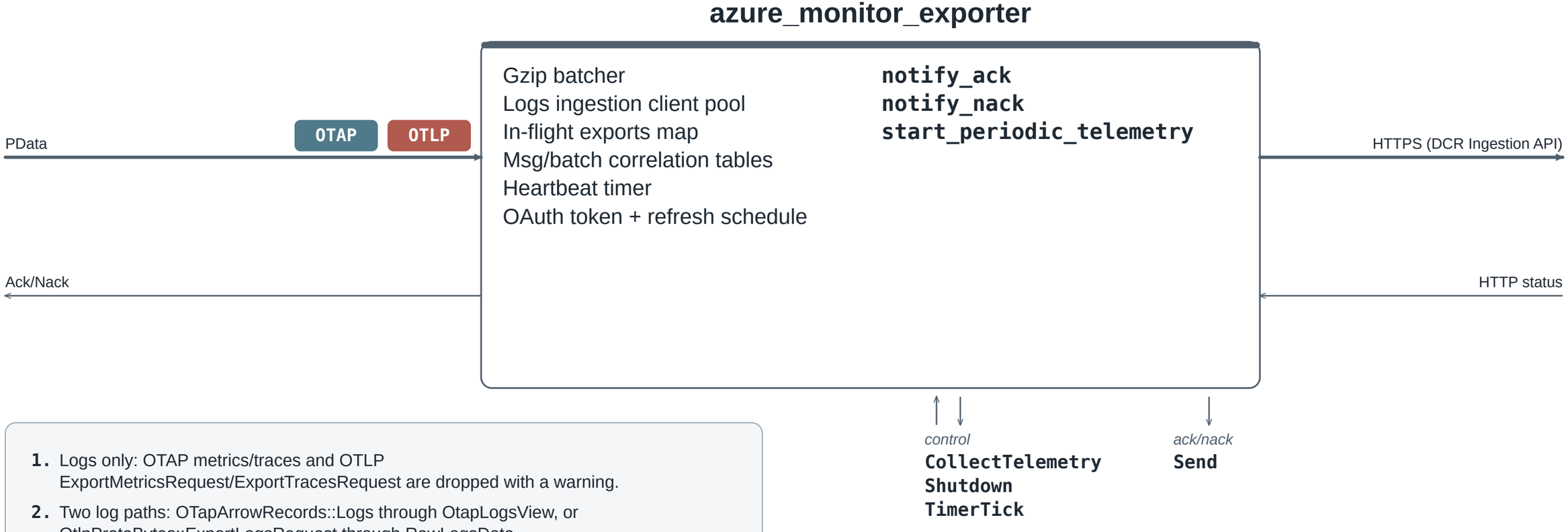
1. Logs and traces are filtered with their configured OTAP filter definitions.
2. Metrics currently pass through the converted OTAP Arrow records unchanged.
3. Inputs are decoded from transport-optimized ids before filters run.
4. Outputs are emitted as OTAP Arrow records even when the input arrived as OTLP bytes.

azure_monitor_exporter

urn:microsoft:exporter:azure_monitor

Sends OpenTelemetry logs to Azure Monitor via the Data Collection Rules (DCR) Logs Ingestion API; metrics and traces are dropped with a warning.

api.dcr_endpoint	String
api.dcr	String
api.stream_name	String
api.schema	SchemaConfig
api.gzip_compression_level	u32
auth.method	ManagedIdentity | Development
auth.client_id	Option<String>
auth.scope	String
heartbeat	HeartbeatConfig



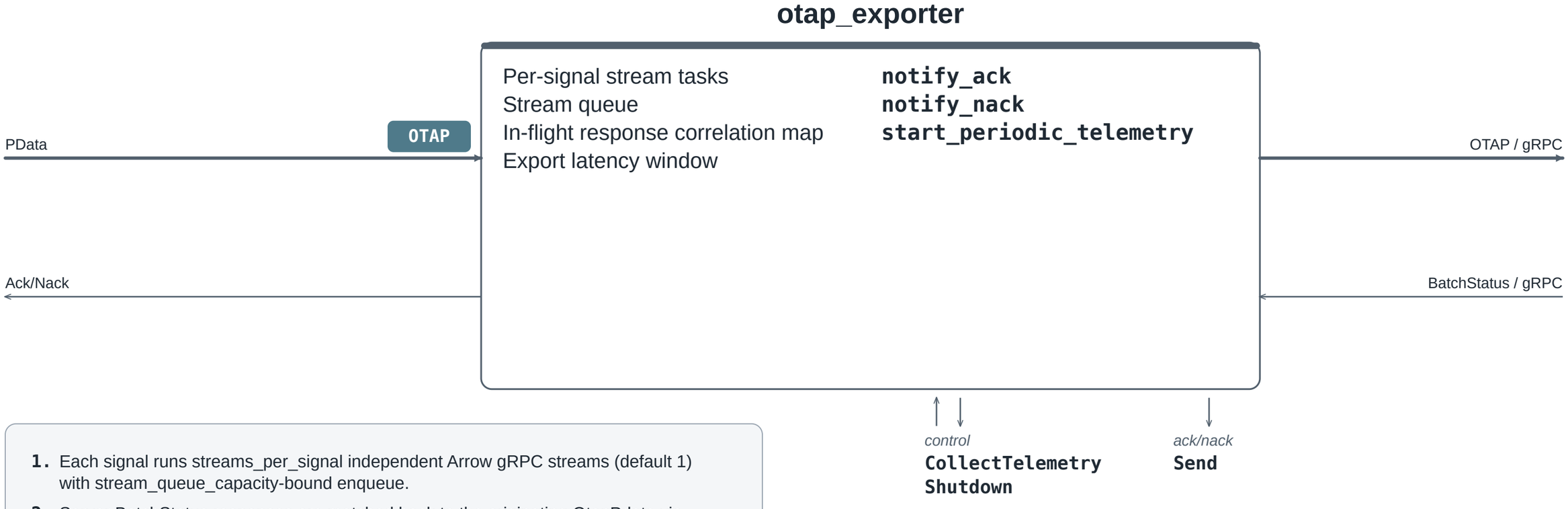
1. Logs only: OTAP metrics/traces and OTLP ExportMetricsRequest/ExportTracesRequest are dropped with a warning.
2. Two log paths: OTapArrowRecords::Logs through OtapLogsView, or OtlpProtoBytes::ExportLogsRequest through RawLogsData.
3. MAX_IN_FLIGHT_EXPORTS = 16 concurrent HTTP POSTs; backpressure is enforced by the in-flight slot pool.

otap_exporter

urn:otel:exporter:otap

Streams OTAP record batches over per-signal Arrow gRPC streams (logs / metrics / traces) with response-correlated ack/nack.

grpc	GrpcClientSettings
compression_method	Option<CompressionMethod>
arrow	ArrowConfig
stream_queue_capacity	usize
streams_per_signal	usize
timeout	Option<Duration>



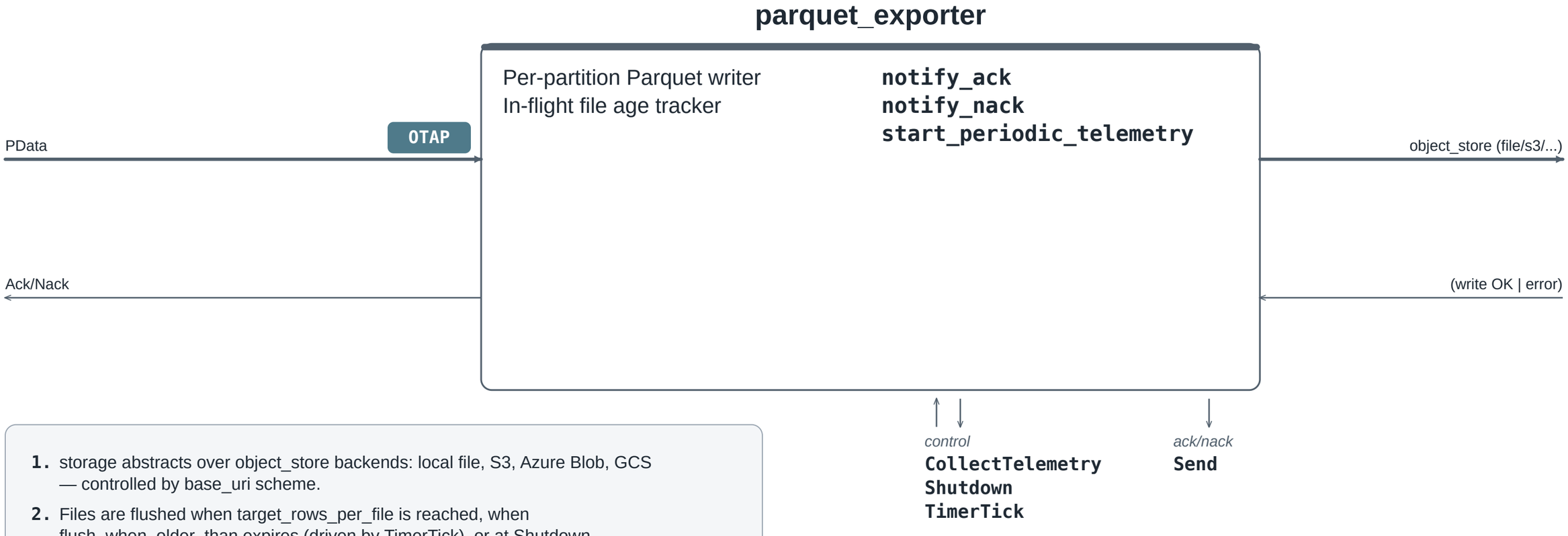
1. Each signal runs streams_per_signal independent Arrow gRPC streams (default 1) with stream_queue_capacity-bound enqueue.
2. Server BatchStatus responses are matched back to the originating OtapPdata via a correlation queue; OK -> notify_ack, anything else -> notify_nack.
3. arrow.compression independently zstd-compresses Arrow IPC frames in addition to gRPC compression (double compression by default).

parquet_exporter

urn:otel:exporter:parquet

Writes OTAP Arrow record batches as partitioned Parquet files to a configured object-store backend.

storage	StorageType
partitioning_strategies	Option<Vec<PartitioningStrategy>>
writer_options.target_rows_per_file	Option<usize>
writer_options.flush_when_older_than	Option<Duration>



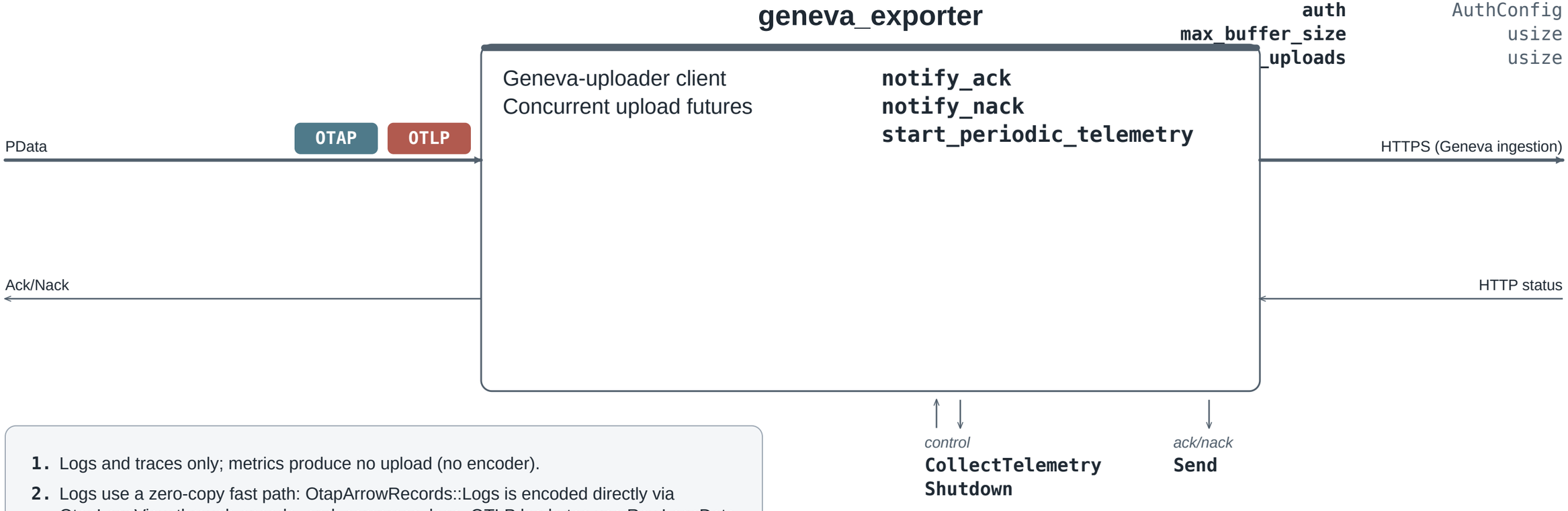
1. storage abstracts over object_store backends: local file, S3, Azure Blob, GCS — controlled by base_uri scheme.
2. Files are flushed when target_rows_per_file is reached, when flush_when_older_than expires (driven by TimerTick), or at Shutdown.
3. PartitioningStrategy::SchemaMetadata extracts named keys from Arrow schema metadata to choose the destination partition.

geneva_exporter

urn:microsoft:exporter:geneva

Encodes and uploads OpenTelemetry logs and traces to Microsoft Geneva via the geneva-uploader client; metrics are not yet supported.

endpoint	String
environment	String
account	String
namespace	String
region	String
config_major_version	u32
tenant	String
role_name	String
role_instance	String
auth	AuthConfig
max_buffer_size	usize
_uploads	usize



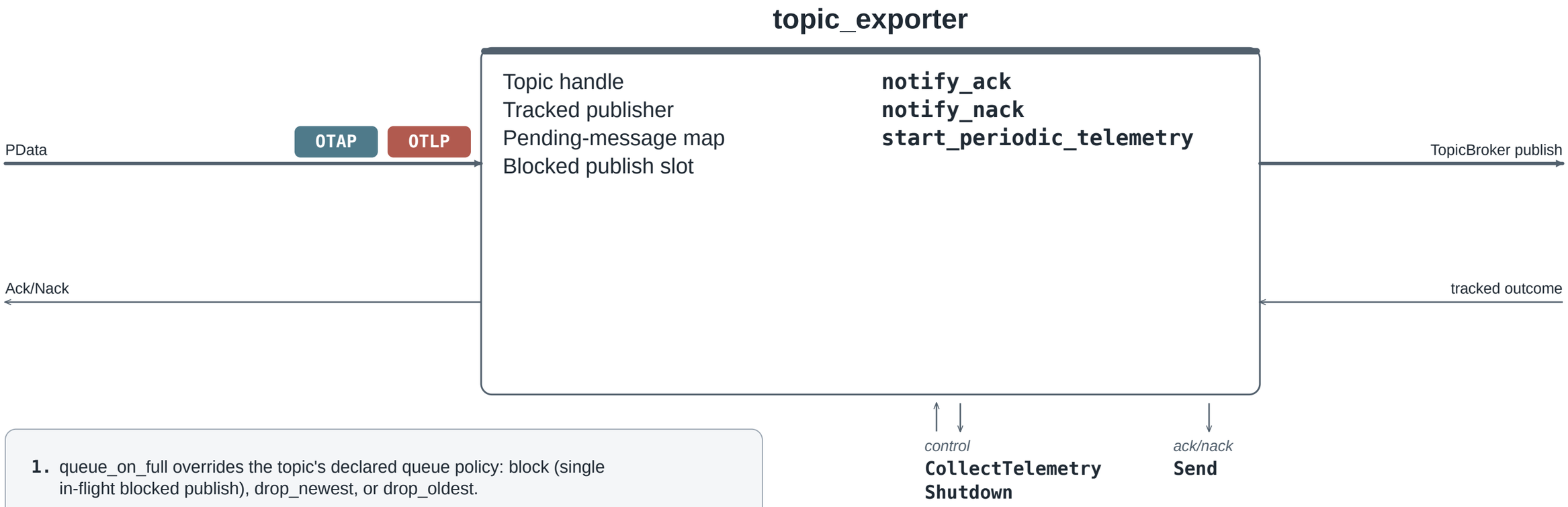
1. Logs and traces only; metrics produce no upload (no encoder).
2. Logs use a zero-copy fast path: OtapArrowRecords::Logs is encoded directly via OtapLogsView through encode_and_compress_logs. OTLP log bytes use RawLogsData over the same path.
3. Traces fall back to the OTLP-bytes path: OTAP traces are transcoded to OTLP protobuf, then uploaded.

topic_exporter

`urn:otel:exporter:topic`

Publishes pdata into an in-process TopicBroker, optionally tracking each publish for end-to-end ack/nack bridging.

`topic`	`TopicName`
`queue_on_full`	`Option<TopicQueueOnFullPolicy>`



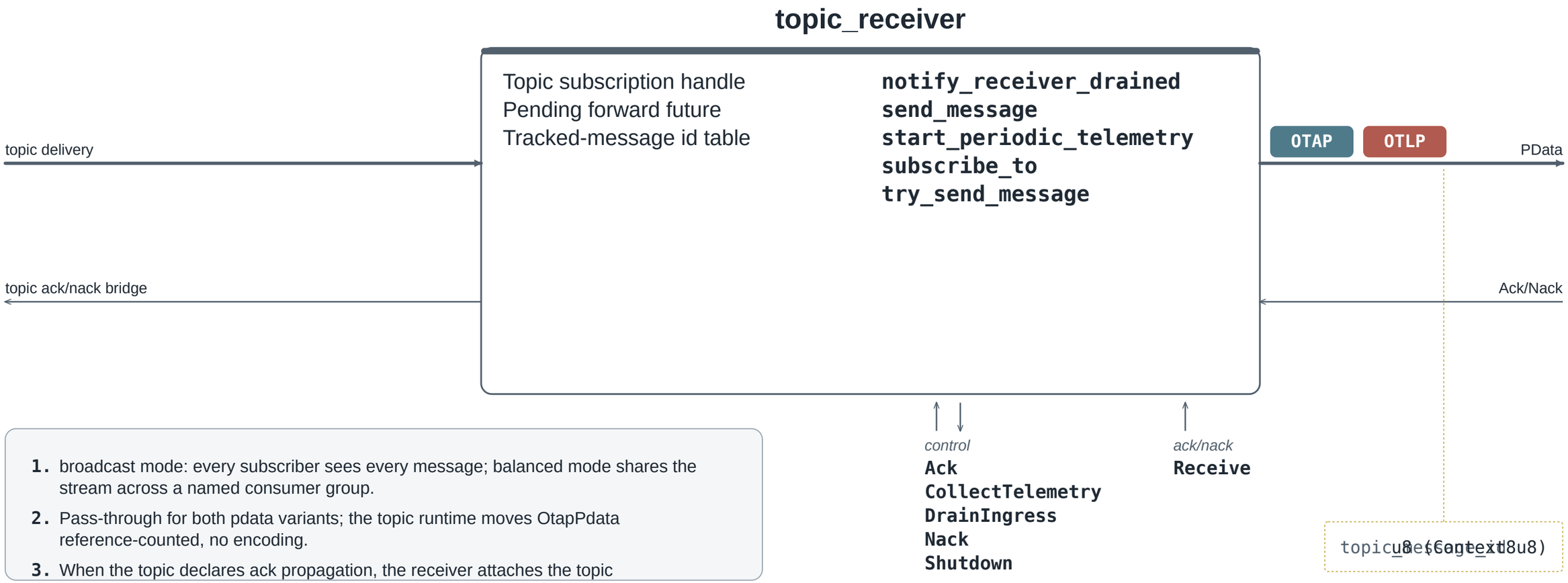
1. queue_on_full overrides the topic's declared queue policy: block (single in-flight blocked publish), drop_newest, or drop_oldest.
2. When the topic declares Auto ack propagation, each publish is tracked; the resolved TrackedPublishOutcome (Ack / Nack / TimedOut / TopicClosed) bridges back as notify_ack / notify_nack to upstream.
3. Pass-through for both pdata variants; payloads are not transformed before publishing.

topic_receiver

urn:otel:receiver:topic

Subscribes to an in-process TopicBroker stream and forwards each delivery downstream as pdata, optionally bridging acks back to the topic runtime.

topic	TopicName
subscription.mode	broadcast | balanced
subscription.group	SubscriptionGroupName

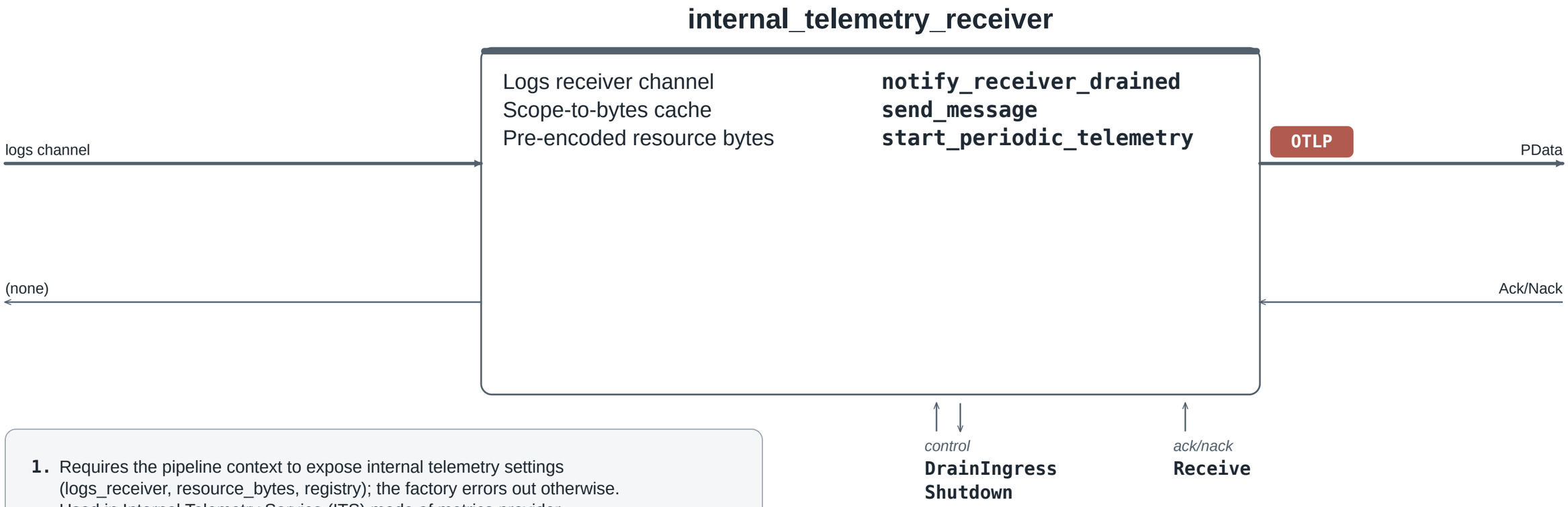


1. broadcast mode: every subscriber sees every message; balanced mode shares the stream across a named consumer group.
2. Pass-through for both pdata variants; the topic runtime moves OtapPdata reference-counted, no encoding.
3. When the topic declares ack propagation, the receiver attaches the topic message_id as 1-byte calldata so downstream Ack/Nack control messages bridge back into the topic runtime.

internal_telemetry_receiver

`urn:otel:receiver:internal_telemetry`

Drains the in-process logging channel and emits each event as an OTLP ExportLogsRequest payload.



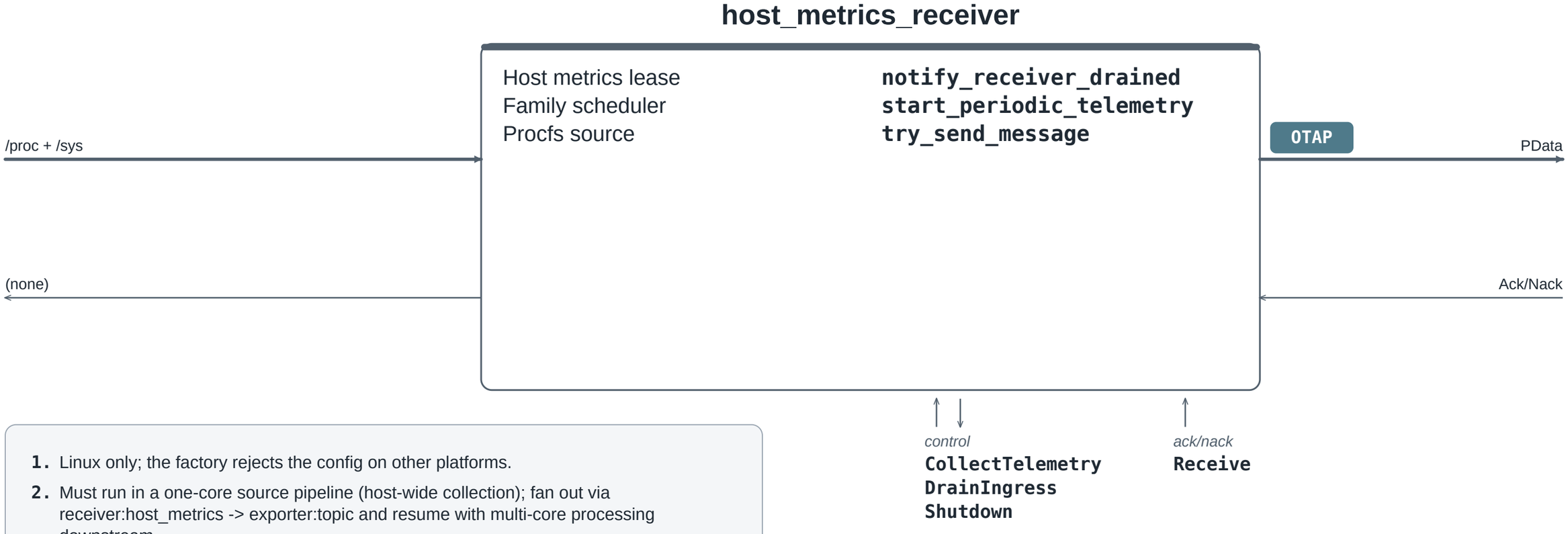
1. Requires the pipeline context to expose internal telemetry settings (logs_receiver, resource_bytes, registry); the factory errors out otherwise. Used in Internal Telemetry Service (ITS) mode of metrics provider.
2. Each ObservedEvent::Log is encoded into an OTLP ExportLogsRequest via encode_export_logs_request and sent downstream. Engine events are not forwarded today.

host_metrics_receiver

`urn:otel:receiver:host_metrics`

Periodically scrapes Linux /proc + /sys host metrics and emits OTAP record batches on a local schedule.

collection_interval	Duration
initial_delay	Duration
root_path	Option<PathBuf>
host_view	HostViewConfig
families	FamiliesConfig



1. Linux only; the factory rejects the config on other platforms.
2. Must run in a one-core source pipeline (host-wide collection); fan out via receiver:host_metrics -> exporter:topic and resume with multi-core processing downstream.
3. FamilyScheduler picks the next due metric family; scrape_due returns a snapshot encoded directly to OTAP record batches.

otap_receiver

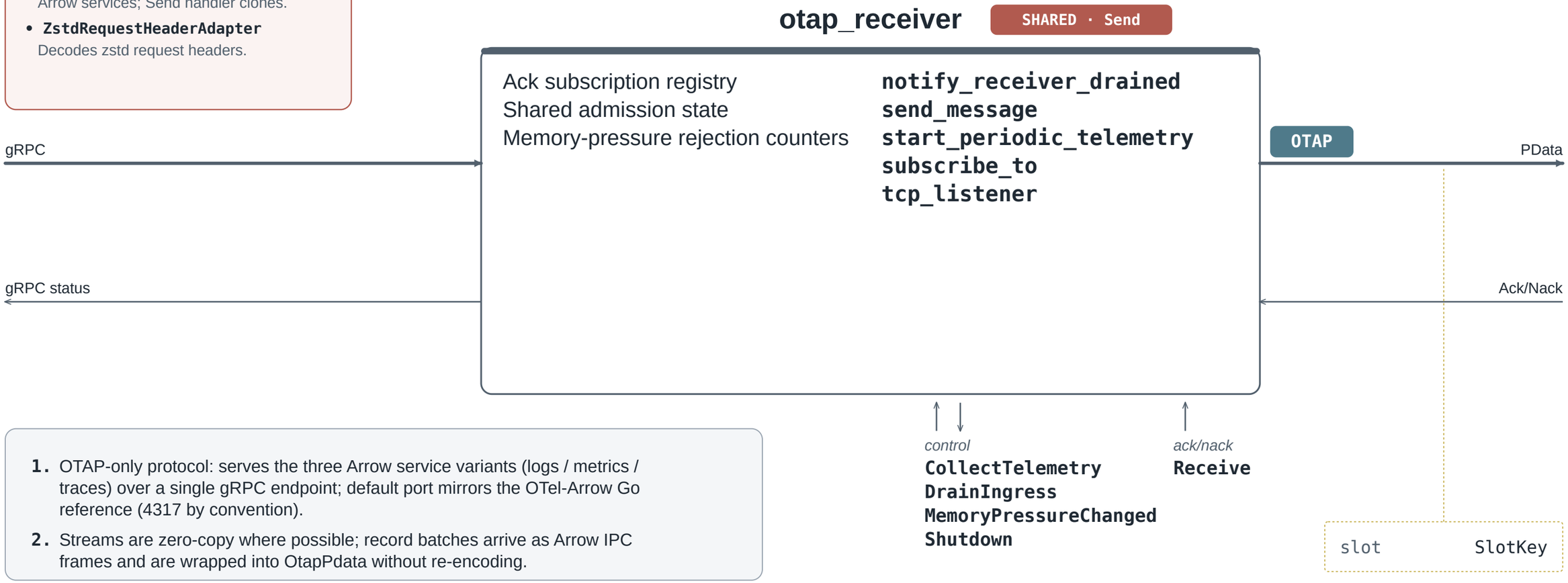
urn:otel:receiver:otap

Ingests OTel-Arrow gRPC streams (logs / metrics / traces) and emits decoded OTAP record batches.

listening_addr	SocketAddr
compression_method	Option<CompressionMethod>
response_stream_channel_size	usize
max_concurrent_requests	usize
max_concurrent_requests_per_stream	NonZeroUsize
wait_for_result	bool
timeout	Option<Duration>
tls	Option<TlsServerConfig>

Foreign entities (Send refs)

- **Tonic gRPC server**
  Arrow services; Send handler clones.
- **ZstdRequestHeaderAdapter**
  Decodes zstd request headers.



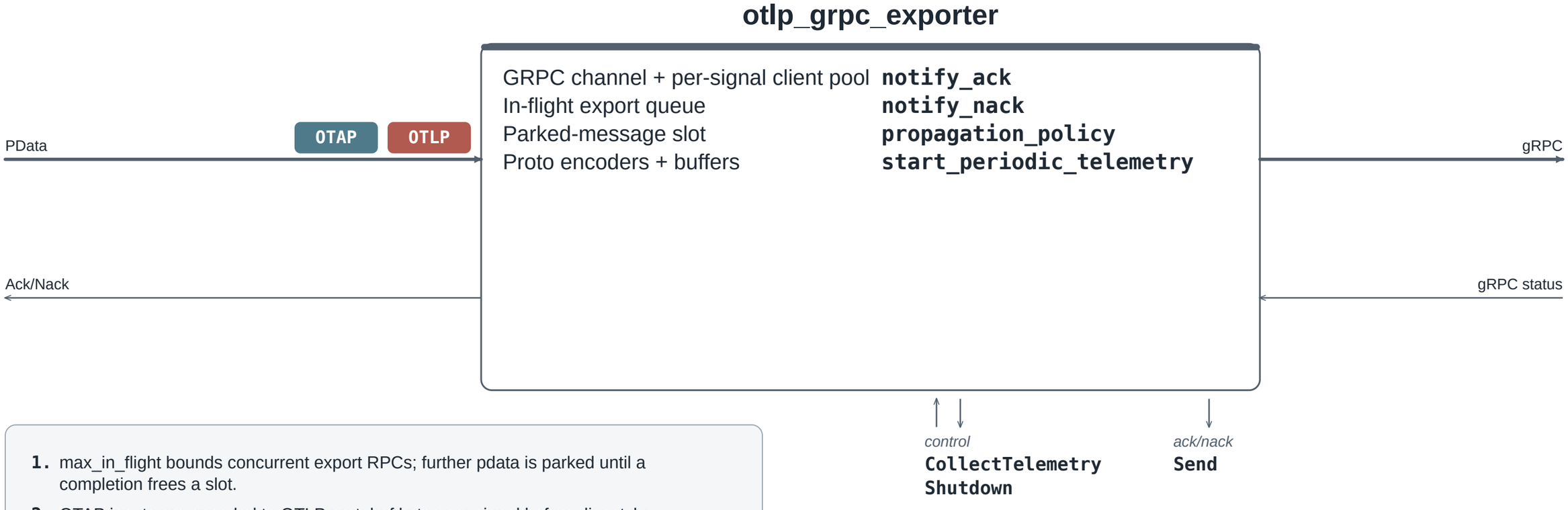
1. OTAP-only protocol: serves the three Arrow service variants (logs / metrics / traces) over a single gRPC endpoint; default port mirrors the OTel-Arrow Go reference (4317 by convention).
2. Streams are zero-copy where possible; record batches arrive as Arrow IPC frames and are wrapped into OtapPdata without re-encoding.
3. wait_for_result holds the per-message response on the gRPC stream until the immediate downstream Acks/Nacks; the default is false (matches the Go

otlp_grpc_exporter

`urn:otel:exporter:otlp_grpc`

Sends OTLP/gRPC exports over a pooled tonic channel with bounded in-flight concurrency.

grpc	GrpcClientSettings
max_in_flight	usize



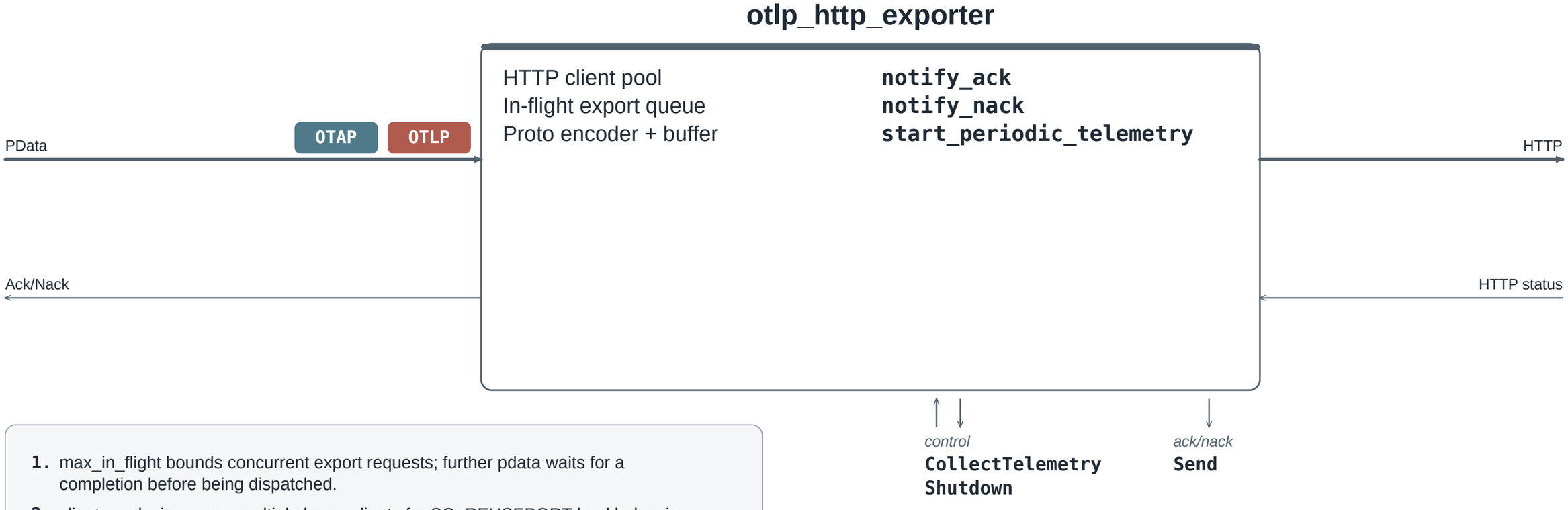
1. max_in_flight bounds concurrent export RPCs; further pdata is parked until a completion frees a slot.
2. OTAP inputs are encoded to OTLP protobuf bytes per signal before dispatch; OTLP inputs go straight to the wire.
3. Successful RPC -> notify_ack with the saved payload; RPC error -> notify_nack with the error string.

otlp_http_exporter

`urn:otel:exporter:otlp_http`

Sends OTLP/HTTP exports over a pooled hyper client with bounded in-flight concurrency.

http	HttpClientSettings
endpoint	String
logs_endpoint	Option<String>
metrics_endpoint	Option<String>
traces_endpoint	Option<String>
max_response_body_length	usize
client_pool_size	NonZeroUsize
max_in_flight	usize



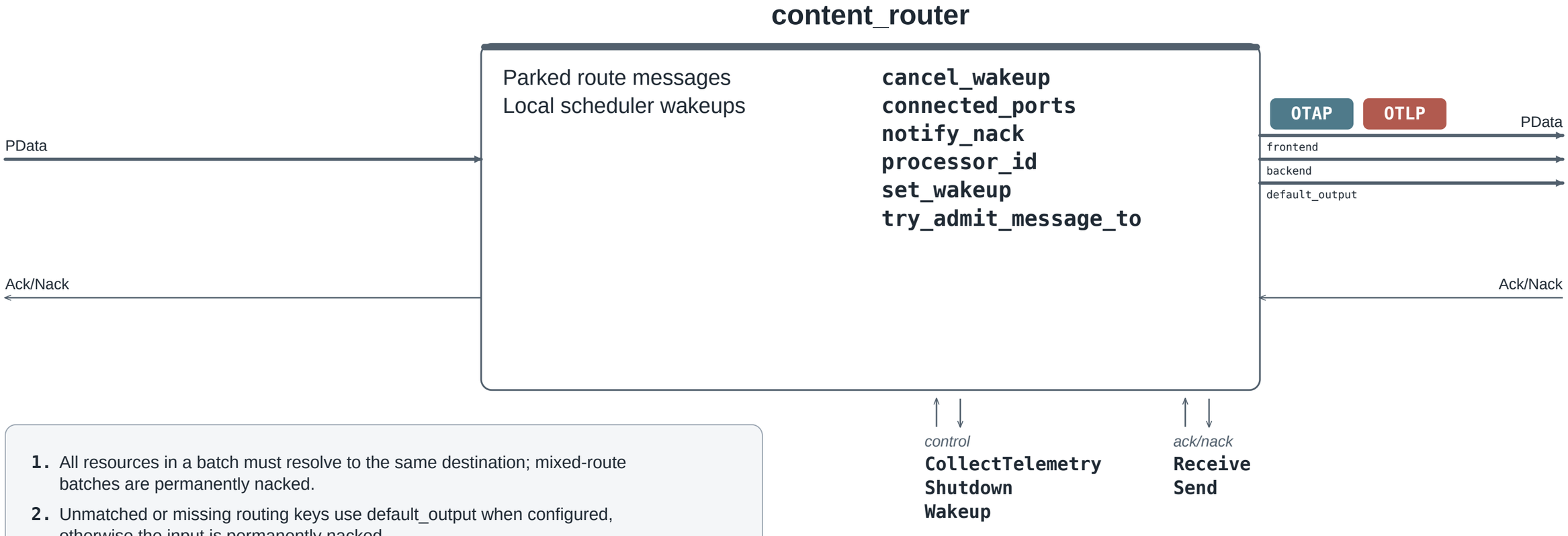
1. max_in_flight bounds concurrent export requests; further pdata waits for a completion before being dispatched.
2. client_pool_size opens multiple hyper clients for SO_REUSEPORT load balancing across receiver instances.
3. Per-signal endpoints (logs / metrics / traces) override the base endpoint when set; otherwise /v1/<signal> is appended automatically.

content_router

`urn:otel:processor:content_router`

Routes each batch to one named output port using a resource attribute value.

routing_key	RoutingKeyExpr
routes	HashMap<String, String>
default_output	Option<String>
case_sensitive	bool
admission_policy	SelectedRouteAdmissionPolicy



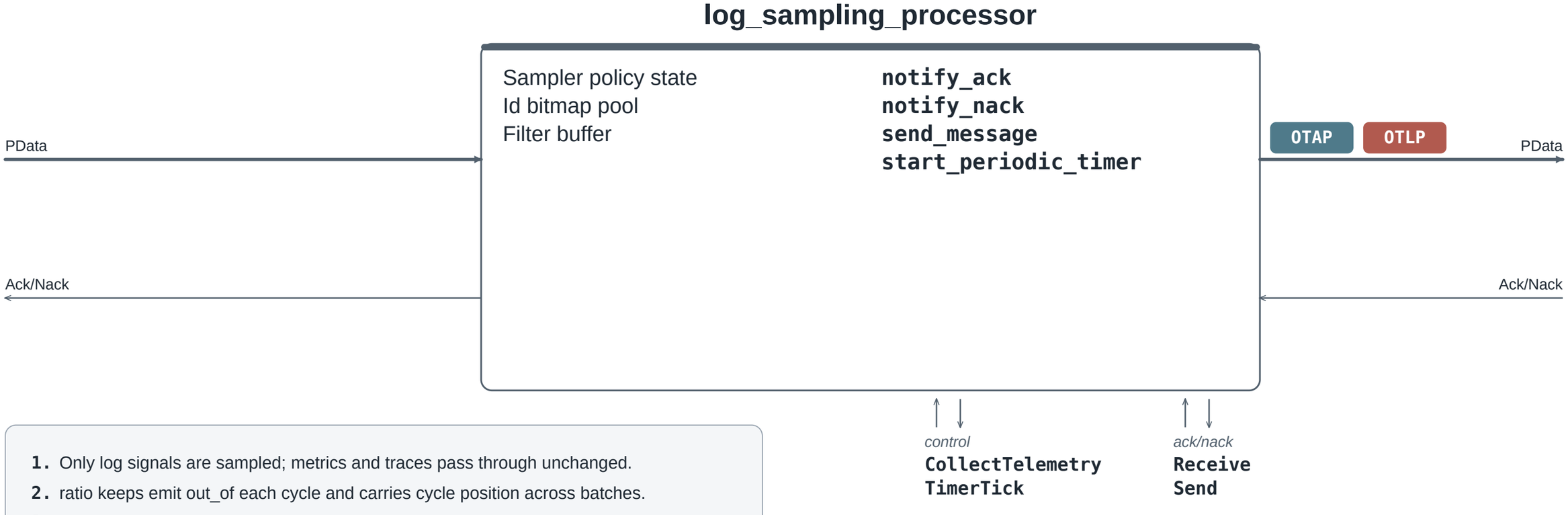
1. All resources in a batch must resolve to the same destination; mixed-route batches are permanently nacked.
2. Unmatched or missing routing keys use default_output when configured, otherwise the input is permanently nacked.
3. The router emits the original pdata format; Arrow metrics and traces may be converted only for route inspection.

log_sampling_processor

`urn:otel:processor:log_sampling`

Reduces log volume with either fixed-ratio sampling or a per-window item budget.

policy	Policy



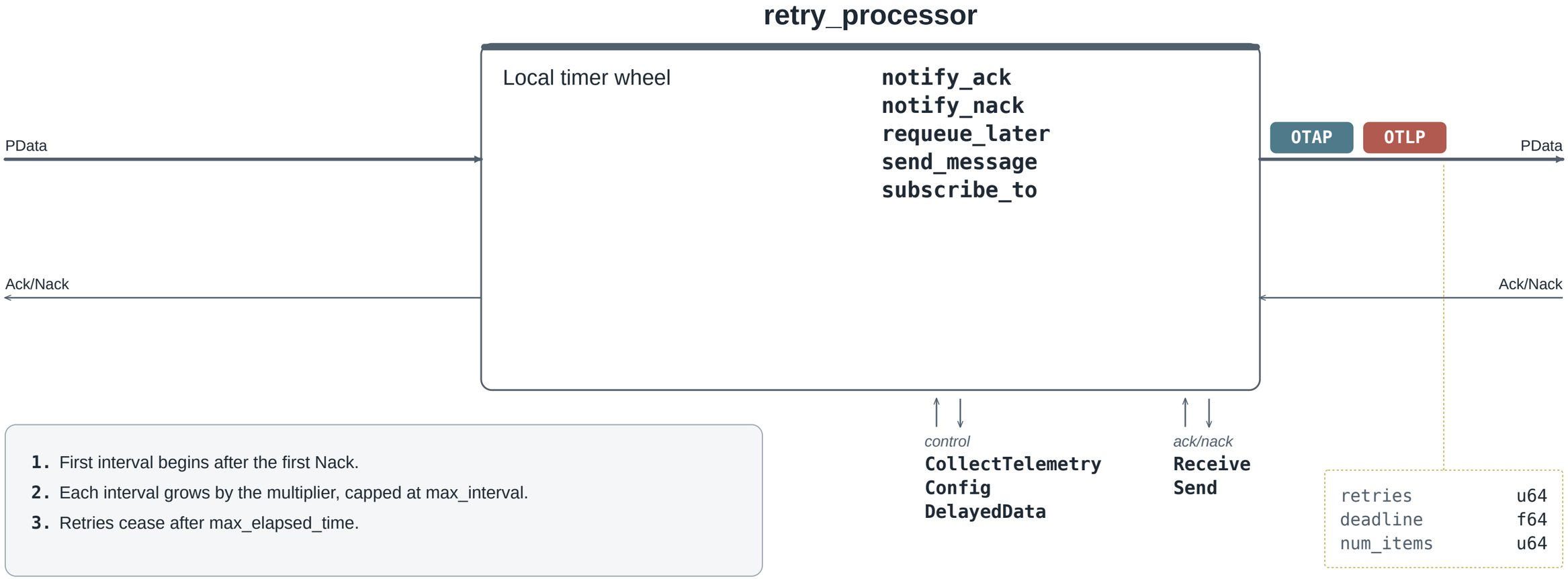
1. Only log signals are sampled; metrics and traces pass through unchanged.
2. ratio keeps emit out_of each cycle and carries cycle position across batches.
3. zip keeps at most max_items per interval and resets its count on TimerTick.
4. Empty sampled results are Acked upstream instead of sending an empty batch.
5. Filtered log outputs are emitted as OTAP Arrow records.

retry_processor

urn:otel:processor:retry

Inline retry with exponential backoff.

initial_interval	Duration
max_interval	Duration
max_elapsed_time	Duration
multiplier	f64



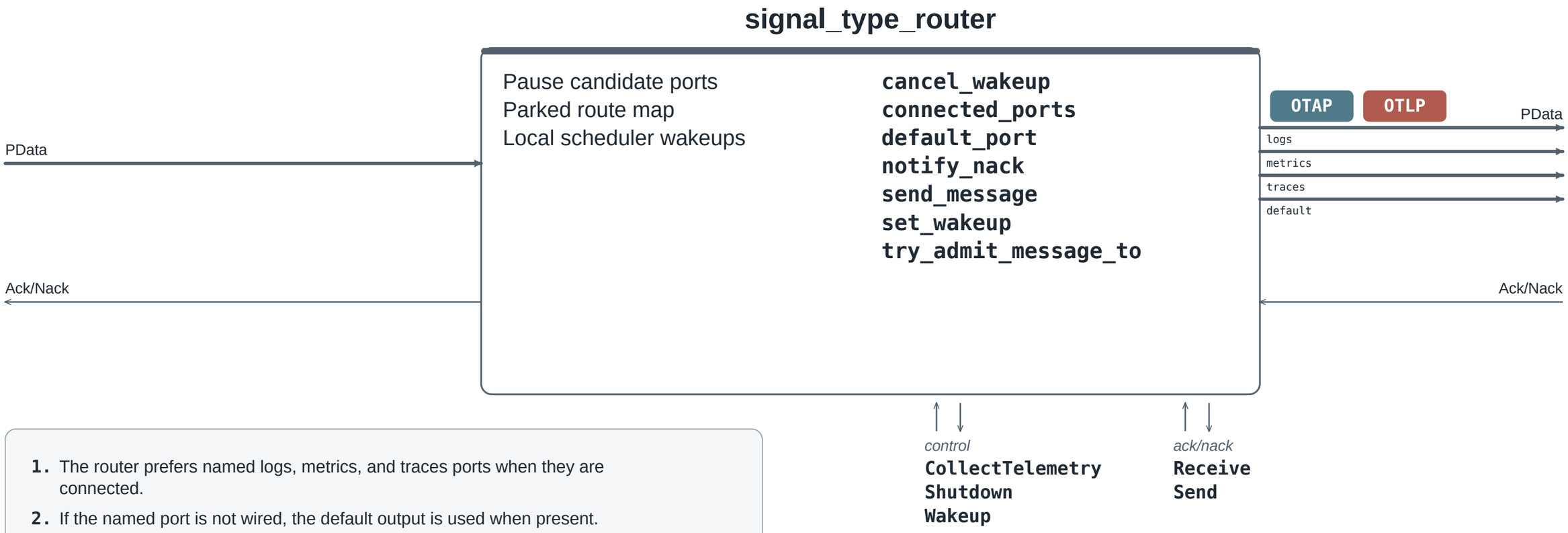
1. First interval begins after the first Nack.
2. Each interval grows by the multiplier, capped at max_interval.
3. Retries cease after max_elapsed_time.

signal_type_router

`urn:otel:processor:type_router`

Routes pdata to logs, metrics, or traces output ports according to signal type.

admission_policy `SelectedRouteAdmissionPolicy`



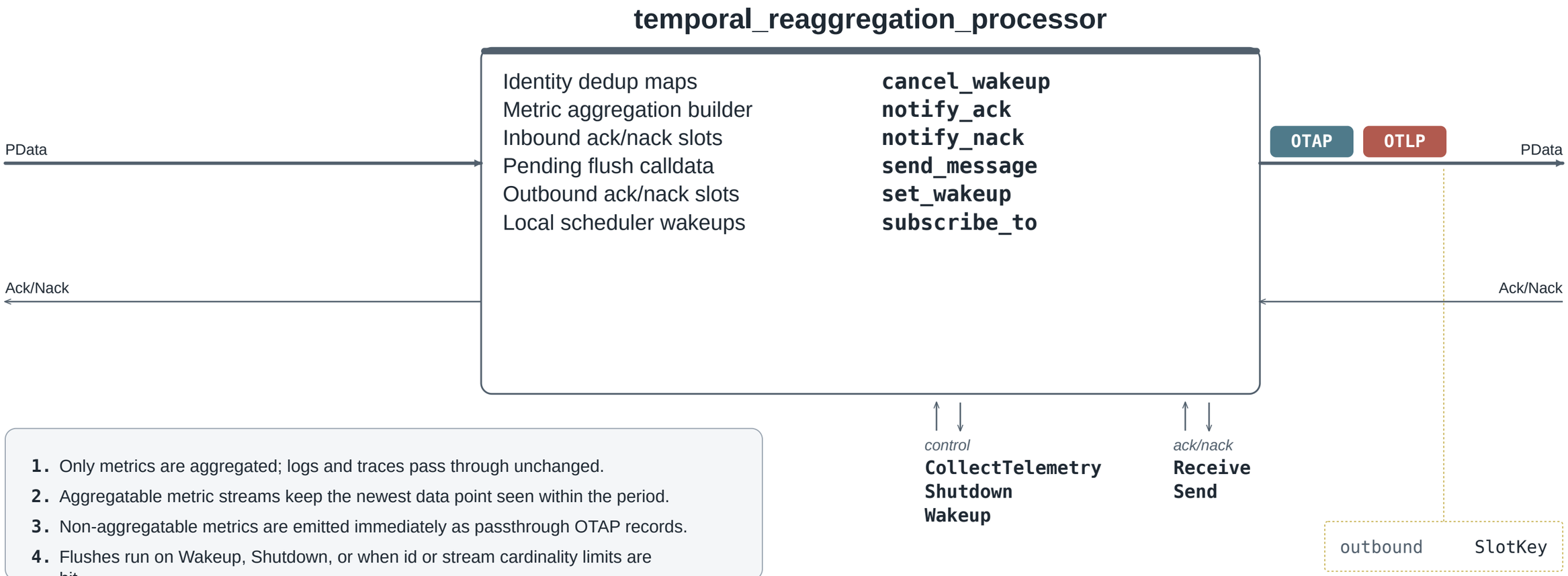
1. The router prefers named logs, metrics, and traces ports when they are connected.
2. If the named port is not wired, the default output is used when present.
3. Selected-route Full can be nacked immediately or parked by backpressure policy.
4. Parked routes are retried through local Wakeup scheduling and nacked during

temporal_reaggregation_processor

`urn:otel:processor:temporal_reaggregation`

Reaggregates delta metrics over a configured period and flushes lower-frequency OTAP batches.

period	Duration
inbound_request_limit	NonZeroUsize
outbound_request_limit	NonZeroUsize
max_stream_cardinality	NonZeroU16



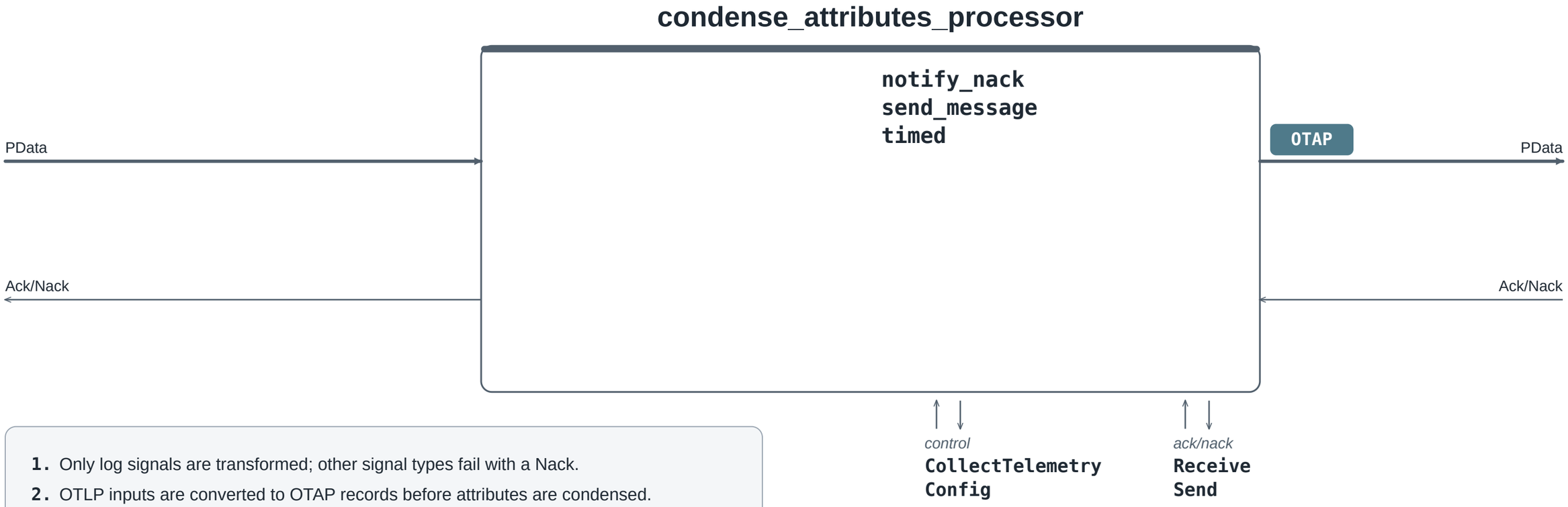
1. Only metrics are aggregated; logs and traces pass through unchanged.
2. Aggregatable metric streams keep the newest data point seen within the period.
3. Non-aggregatable metrics are emitted immediately as passthrough OTAP records.
4. Flushes run on Wakeup, Shutdown, or when id or stream cardinality limits are hit.
5. Inbound Ack waits until every associated passthrough and flushed outbound is

condense_attributes_processor

`urn:otel:processor:condense_attributes`

Condenses selected log attributes into one destination attribute on OTAP output records.

destination_key	`String`
delimiter	`char`
source_keys	`Option<HashSet<String>>`
exclude_keys	`Option<HashSet<String>>`



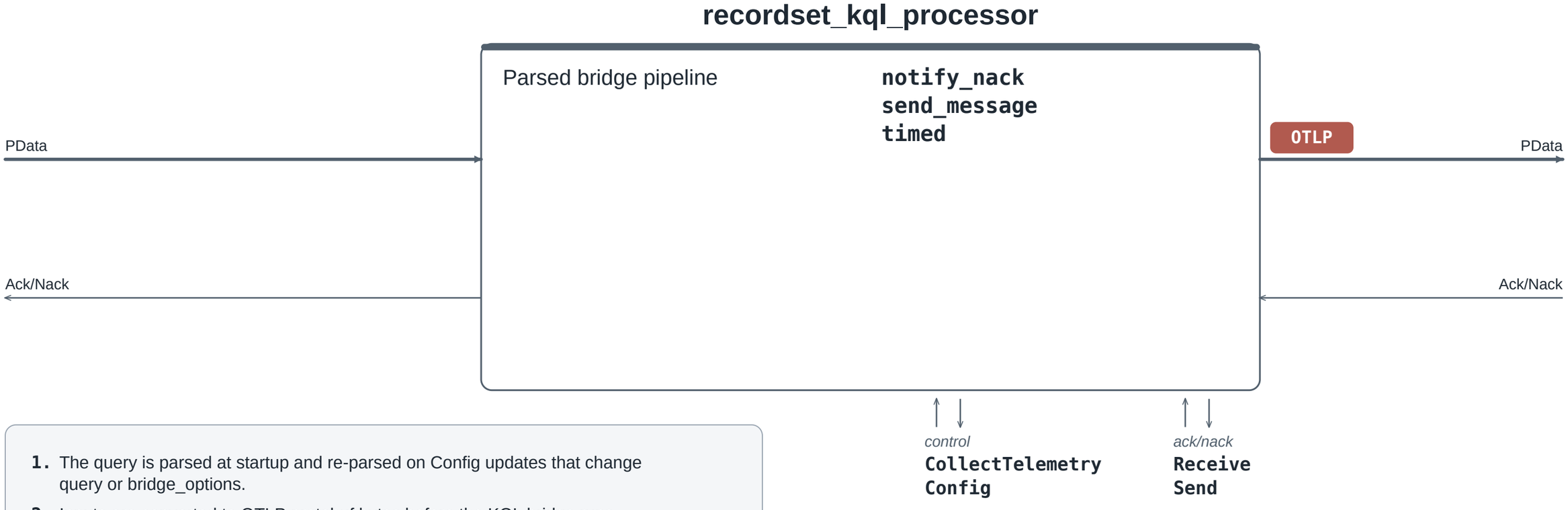
1. Only log signals are transformed; other signal types fail with a Nack.
2. OTLP inputs are converted to OTAP records before attributes are condensed.
3. source_keys and exclude_keys are mutually exclusive; if neither is set, all attributes are condensed.
4. Existing destination_key attributes are removed before the new condensed attribute is written.

recordset_kql_processor

`urn:microsoft:processor:recordset_kql`

Runs a parsed RecordSet KQL pipeline over OTLP log records carried by OTAP pdata.

query	`String`
bridge_options	`Option<serde_json::Value>`



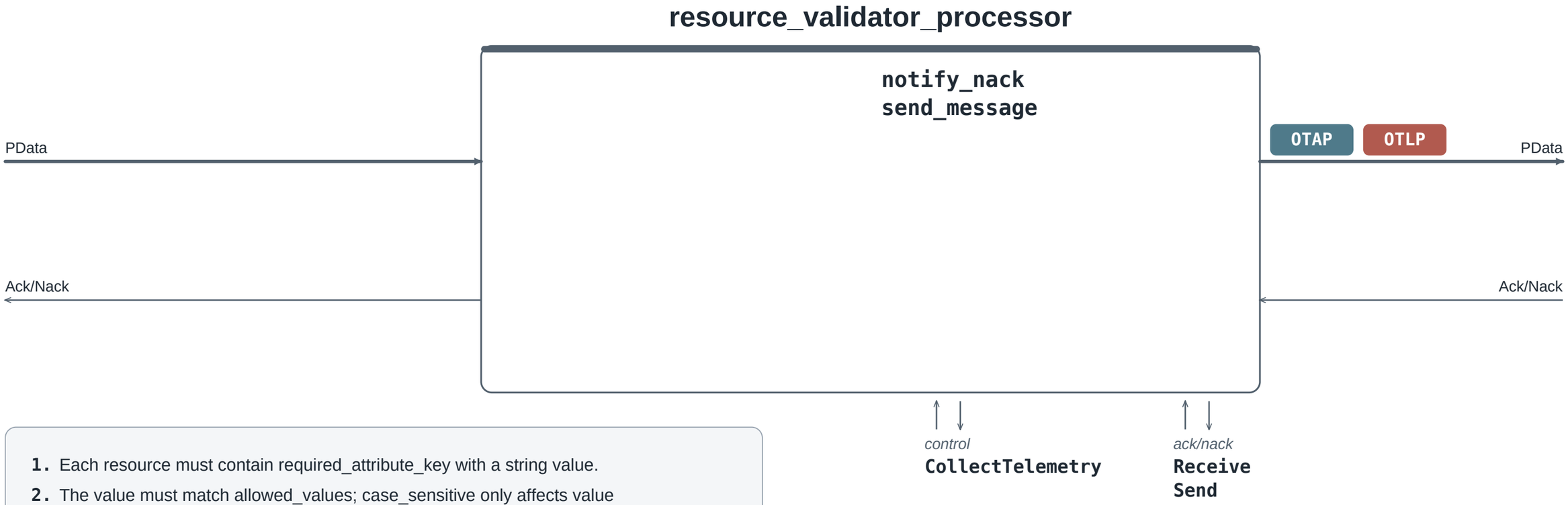
1. The query is parsed at startup and re-parsed on Config updates that change query or bridge_options.
2. Inputs are converted to OTLP protobuf bytes before the KQL bridge runs.
3. Only logs are implemented; metrics and traces return internal errors.
4. Successful outputs are emitted as OTLP bytes in the original pdata context.

resource_validator_processor

`urn:otel:processor:resource_validator`

Validates required resource attributes and permanently Nacks data that fails policy.

required_attribute_key	String
allowed_values	Vec<String>
case_sensitive	bool



1. Each resource must contain required_attribute_key with a string value.
2. The value must match allowed_values; case_sensitive only affects value matching.
3. Validation failures produce permanent Nacks instead of forwarding the data.
4. OTAP log records are validated directly; OTAP metrics and traces are converted to OTLP for validation.